

Konzeption und Evaluation eines
Multi-Agenten-Systems zur automatisierten Migration
von imperativer LO-VC Logik zu deklarativer AVC
Syntax am Beispiel der msg systems ag

Torben Dörrer

30. August 2026

Inhaltsverzeichnis

Abkürzungsverzeichnis	iv
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung und Forschungsfragen	2
1.3 Methodisches Vorgehen und Forschungsdesign	3
1.4 Abgrenzung der Arbeit	4
1.5 Aufbau der Arbeit	4
2 Theoretische Grundlagen	6
2.1 Domänenspezifische Grundlagen: SAP Variantenkonfiguration	6
2.1.1 Einführung in die logische Architektur der Variantenkonfiguration	6
2.1.2 Die klassische Variantenkonfiguration (LO-VC) im Kontext der Systemtransformation	7
2.1.3 Die moderne Architektur (AVC) und das Constraint Paradigma	8
2.1.4 Strukturelle Härte der Migration: Vorbedingungen vs. Constraint-Tabellen	9
2.2 Generative künstliche Intelligenz im Software Engineering	11
2.2.1 Large Language Models für Quellcode	11
2.2.2 Limitationen monolithischer Ansätze	12
2.3 Multi-Agenten-Systeme (MAS)	14
2.3.1 Definition und kognitive Architekturen	14
2.3.2 Kollaborative Agenten-Muster	16
2.3.3 Entwicklungs-Frameworks für MAS	17
3 Related Work	19
4 Experteninterview	22
4.1 Methodisches Setup und Durchführung	22
4.2 Ergebnisse der Befragung	24
4.2.1 Kognitive Altlasten und historischer Code	24
4.2.2 Paradigmenwechsel und syntaktisches Umdenken	24

4.2.3	Fragmentierung manueller Prozessschritte	24
4.2.4	Limitierungen bestehender Standard-Tools	25
4.2.5	Manueller Aufwand und Fehleranfälligkeit im Testprozess	25
4.2.6	Zeitliche Metriken und Identifikation von Flaschenhälsen	25
4.3	Implikationen für die Systemarchitektur (Ideales Zielbild)	26
4.3.1	Funktionale Anforderungen aus der Expertenperspektive	26
4.3.2	Architektonische Paradigmen und Beratungsansatz	27
4.3.3	Quantitative Zielvorgaben für die Systemvalidierung	27
5	Machbarkeitsanalyse und Selbstversuch	29
5.1	Technologischer Realitätsabgleich und empirische Scope-Definition	29
5.1.1	Evaluation der funktionalen Praxisanforderungen	30
5.1.2	Evaluation der architektonischen Paradigmen	30
5.1.3	Synthese des finalen System-Scopes	31
5.2	Methodisches Setup und Durchführung	31
5.3	Ergebnisse des Selbstversuchs und Ableitung der kognitiven Architektur . .	33
5.3.1	Kognitive Phasen der manuellen Migration	33
5.3.2	Ableitung der V1-Architektur und System-Prompts	35
6	Systemdesign und Prototypenentwicklung	37
6.1	Technologieauswahl und Infrastruktur	37
6.1.1	Orchestrierungs-Framework: CrewAI	37
6.1.2	Infrastruktur und Modellintegration: SAP AI Core und liteLLM . .	38
6.2	Iterative Entwicklung des Prototyps (Design Science Research)	39
6.2.1	Zyklus 1: Limitierungen einer rein agentenbasierten Architektur . .	39
6.2.2	Zyklus 2: Code-Refaktorisierung, Wissensbasis und strukturelle Hal- luzinationen	40
6.2.3	Zyklus 3: Regelbasierte Vorverarbeitung und Domänenunabhängigkeit	42
6.2.4	Zyklus 4: Die finale neuro-symbolische Architektur und funktionale Abgrenzung	43
7	Qualitative Evaluation	46
7.1	Methodisches Setup und Durchführung	46
7.2	Ergebnisse der Experten-Evaluation	48
7.2.1	Nachvollziehbarkeit (Explainability & Traceability)	48
7.2.2	Semantische Äquivalenz und Logik (Struktur & Architektur)	48
7.2.3	Syntaktische Korrektheit (AVC-Compliance)	50
7.2.4	Praxistauglichkeit und Business Value	50
7.3	Limitierungen der Fallstudie (Threats to Validity)	51

8	Fazit und Ausblick	53
8.1	Zusammenfassung der Arbeit	53
8.2	Beantwortung der Forschungsfragen	54
8.3	Limitierungen der Arbeit	56
8.3.1	Eingeschränkte Datenverfügbarkeit und fehlende Ground Truth . .	56
8.3.2	Domänenkomplexität und Expertenverfügbarkeit	56
8.4	Gelernte Lektionen und zukünftige Forschungsfelder	57
8.4.1	Wissenschaftliche Einordnung: Warum Künstliche Intelligenz? . . .	57
8.4.2	Technische Weiterentwicklung des Artefakts	57
8.4.3	Reevaluation der Modellwahl: Code-LLMs für die reine Code-Synthese	58
8.5	Schlusswort	58
	Literaturverzeichnis	60
A	Interview- und Evaluationsleitfaden	63
A.1	Leitfaden Experteninterview	63
A.2	Leitfaden Evaluation	65
B	Codebücher der qualitativen Analyse	67
B.1	Codebuch Experteninterview	67
B.2	Codebuch Evaluation	72
C	Dokumentation Selbstversuch	74
D	Beispiel	80
D.1	Beispielhafter JSON Input	80
D.2	Erstes Beispiel eines Ergebnisses der V1-Architektur	82
D.3	Erstes Beispiel des Ergebnisses der finalen Architektur	83

Abkürzungsverzeichnis

KMAT konfigurierbare Materialstamm

Super BOM konfigurierbare Maximalstückliste

AVC Advanced Variant Configuration

LO-VC Logistics Variant Configuration

KI künstliche Intelligenz

ML Machine Learning

CSP Constraint Satisfaction Problem

LLMs Large Language Models

ICL In-Context Learning

MAS Multi-Agenten-Systems

DSR Design Science Research

POC Proof of Concept

Kapitel 1

Einleitung

1.1 Motivation und Problemstellung

Produzierende Unternehmen stehen aktuell im SAP Ökosystem vor der Herausforderung, ihre IT-Landschaften vom vorherigen SAP ERP auf das moderne System SAP S/4HANA zu migrieren (vgl. Kapitel 2.1). Ein kritischer Teilbereich dieser Umstellung ist die Variantenkonfiguration: Die imperative Logik der klassischen *Logistics Variant Configuration* (LO-VC) muss in die deklarative Syntax der *Advanced Variant Configuration* (AVC) überführt werden.

In der Praxis – insbesondere im Beratungsumfeld der *msg systems ag* – erweist sich diese Syntax-Migration als enormer Zeit- und Kostenfaktor. Da es wegen des grundlegenden Paradigmenwechsels keine einfachen 1:1-Übersetzer gibt, erfolgt die Umstellung oft manuell und ist entsprechend fehleranfällig. Erste Versuche der *msg systems ag*, diesen Prozess mit generativer *künstliche Intelligenz* (KI) zu automatisieren, blieben bislang hinter den Erwartungen zurück. Einfache Abfragen über Sprachmodelle wie ChatGPT scheitern an der strukturellen Komplexität und den strikten Syntaxvorgaben der SAP-Constraint-Netzwerke. Bei langen Logikketten verlieren die Modelle den Kontext und erzeugen fehlerhaften Code.

Aus dieser Problematik leitet sich die zentrale Motivation der vorliegenden Arbeit ab. Da sich die komplexe Übersetzungsaufgabe bisher nicht zuverlässig durch monolithische Single-Prompt-Ansätze lösen ließ, wird untersucht, inwiefern der Einsatz eines *Multi-Agenten-Systems* (MAS) zu besseren Ergebnissen führt. Die Ausgangshypothese lautet dabei, dass ein orchestriertes Kollektiv spezialisierter KI-Agenten, die das Migrationsproblem in noch zu definierende, logische Teilaufgaben zerlegen, Entwickler grundlegend dabei unterstützen kann, historischen Code und komplexe Altsysteme schneller zu durchdringen. Durch die gezielte Automatisierung eines spezifischen Teilbereichs der Migration soll so nicht nur die strukturelle Qualität der Zielarchitektur verbessert, sondern letztlich auch ein effizienterer und beschleunigter Gesamtprozess ermöglicht werden.

1.2 Zielsetzung und Forschungsfragen

Das primäre Ziel dieser Masterarbeit ist die theoriegeleitete Konzeption, Implementierung und Evaluation eines *Proof of Concept* (POC) für eine Multi-Agenten-Architektur zur Logik-Migration von LO-VC nach AVC. In enger Kooperation mit der *msg systems ag* wird hierfür zunächst der komplexe Problemraum analysiert, um systematisch herauszuarbeiten, welcher spezifische Teilbereich der Migration im Rahmen dieser Arbeit fokussiert und automatisiert werden soll.

Aufbauend auf dieser Abgrenzung wird iterativ ein Prototyp entwickelt. Dabei soll praxisnah untersucht werden, wie ein Multi-Agenten-System für diesen identifizierten Teilbereich strukturiert sein muss und ob der Einsatz generativer KI den intellektuellen Engpass der manuellen Regelwerks-Transformation auflösen kann. Die abschließende Evaluation des Prototyps dient dazu, den tatsächlichen Mehrwert sowie die technologischen Grenzen dieses Automatisierungsansatzes aufzuzeigen. Die Arbeit wird hierbei von folgender Hauptforschungsfrage geleitet:

Inwiefern lässt sich die Migration von klassischer LO-VC-Logik zu deklarativer AVC-Syntax durch eine Multi-Agenten-Architektur automatisieren, und wo liegen die Grenzen dieses Ansatzes?

Um diese Hauptfrage zu beantworten, werden vier Unter-Forschungsfragen (FF) definiert:

- **FF1 (Problemraum & Baseline):** Welche prozessualen Hürden prägen die manuelle Migration der historischen SAP-Modelle, und wie hoch ist der bisherige zeitliche Aufwand?
- **FF2 (Aufgabendesign):** Wie lassen sich die manuellen Übersetzungsschritte eines menschlichen Entwicklers so strukturieren, dass daraus sinnvolle Rollen und Aufgaben für ein MAS abgeleitet werden können?
- **FF3 (Architekturentwurf & Limitierungen):** Welche technologischen Herausforderungen ergeben sich beim Einsatz generativer KI-Agenten für komplexe Code-Transformationen, und wie muss die Architektur gestaltet werden, um zuverlässige Ergebnisse zu liefern?
- **FF4 (Evaluation & Mehrwert):** Welchen qualitativen Mehrwert bietet der entwickelte Prototyp im direkten Vergleich zur manuellen Baseline, und wo liegen die funktionalen Grenzen für einen vollautonomen Praxiseinsatz?

1.3 Methodisches Vorgehen und Forschungsdesign

Die Methodik dieser Arbeit basiert auf dem Paradigma des Design Science Research (*Design Science Research* (DSR)) nach Hevner et al. (2004). Ziel ist es, nicht nur ein Praxisproblem theoretisch zu analysieren, sondern aktiv eine innovative IT-Lösung (den MAS-Prototyp) dafür zu entwickeln. DSR verbindet dabei praktische Relevanz mit wissenschaftlicher Methodik.

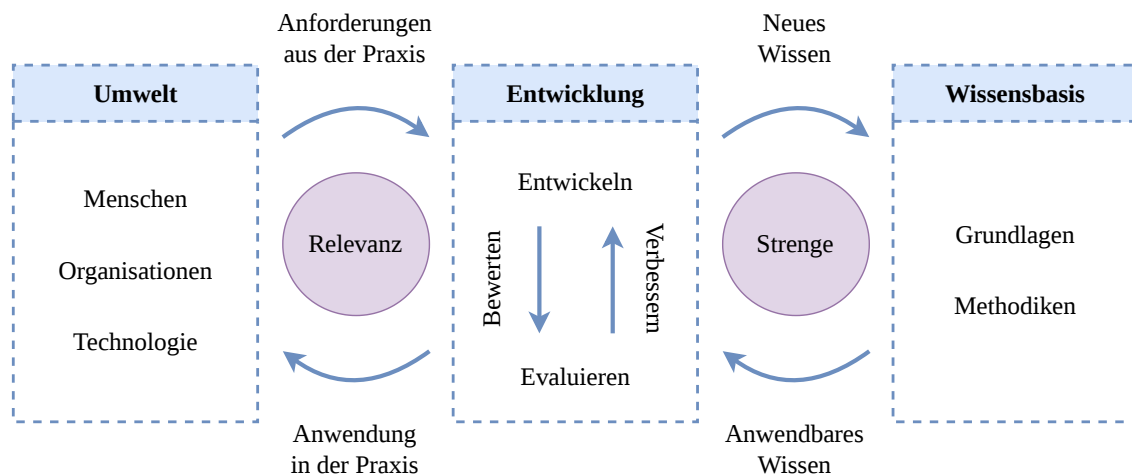


Abbildung 1.1: Das Design Science Research Framework (in Anlehnung an Hevner et al., 2004).

Das Vorgehen gliedert sich in drei Kernphasen, die verschiedene Methoden kombinieren:

1. **Explorative Phase (Problem- und Scope-Definition):** Zunächst wird der Problemraum bei der *msg systems ag* präzise erfasst.
 - *Fallstudie 1 (Expertenbefragung):* Ein semi-strukturiertes Interview ermittelt die manuelle Baseline und die Anforderungen aus der Praxis. Die Auswertung erfolgt mittels qualitativer Inhaltsanalyse nach Mayring.
 - *Praktischer Selbstversuch:* In Anlehnung an empirisches Software Engineering nach Seaman (1999) werden reale LO-VC-Daten systematisch analysiert. So wird auf Code-Ebene geprüft, wie sich die Aufgaben für die spezialisierten KI-Rollen ableiten lassen.
2. **Konstruktionsphase (Build):** Aufbauend auf der ersten Phase wird der Prototyp in mehreren Iterationen entwickelt und verfeinert.
3. **Evaluationsphase (Evaluate):** Um den praktischen Nutzen der Lösung zu belegen, evaluiert ein Fachexperte das System abschließend qualitativ mithilfe der *Think-Aloud-Methode*. Das geführte Interview wird wiederum nach Mayring ausgewertet, um

den generierten Code auf Nachvollziehbarkeit, Semantik und syntaktische Korrektheit zu prüfen.

1.4 Abgrenzung der Arbeit

Diese Arbeit beschränkt sich auf die Konzeption und Entwicklung eines Multi-Agenten-Prototyps für die Logik-Transformation von LO-VC nach AVC im Kontext der *msg systems ag*.

Das Problemfeld der SAP-Migration ist hochkomplex. Eine vollautomatische, produktionsreife Enterprise-Lösung für den gesamten Migrationsprozess wird hier daher nicht angestrebt. Innerhalb des Zeitrahmens einer sechsmonatigen Masterarbeit lassen sich Themen wie Massendatenverarbeitung, vollständige Systemintegration oder vollautomatisiertes Testing nicht sinnvoll abdecken.

Vielmehr dient die Arbeit als Machbarkeitsstudie (POC). Anhand eines isolierten Beispielszenarios wird detailliert untersucht, an welchen Stellen der Einsatz generativer KI im Rahmen eines MAS sinnvoll ist und wo die aktuellen technologischen Grenzen liegen.

Der exakte funktionale Scope wird nicht vorab festgelegt, sondern theoriegeleitet aus der Problem-Exploration (Kapitel 4 und 5) abgeleitet, um den Agenten gezielt dort einzusetzen, wo der prozessuale Leidensdruck am höchsten ist.

1.5 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in acht aufeinander aufbauende Kapitel, deren Inhalte sich wie folgt strukturieren:

Kapitel 1 – Einleitung: Führt in die Problemstellung ein, definiert die Forschungsfragen und skizziert das methodische Design sowie die Abgrenzung der Arbeit.

Kapitel 2 – Theoretische Grundlagen: Legt das technologische Fundament zur SAP-Variantenkonfiguration und beleuchtet die Paradigmen von Multi-Agenten-Systemen.

Kapitel 3 – Related Work: Ordnet die Arbeit durch eine Literaturanalyse in den aktuellen wissenschaftlichen Forschungskontext ein.

Kapitel 4 – Experteninterview: Beschreibt die Durchführung des Experteninterviews und wertet die qualitative Befragung aus, um die manuelle Baseline und die funktionalen Praxisanforderungen zu definieren.

Kapitel 5 – Machbarkeitsanalyse und Selbstversuch: Führt eine kurze Machbarkeitsanalyse der durch das Interview gewonnenen Erkenntnisse für die Anforderungen

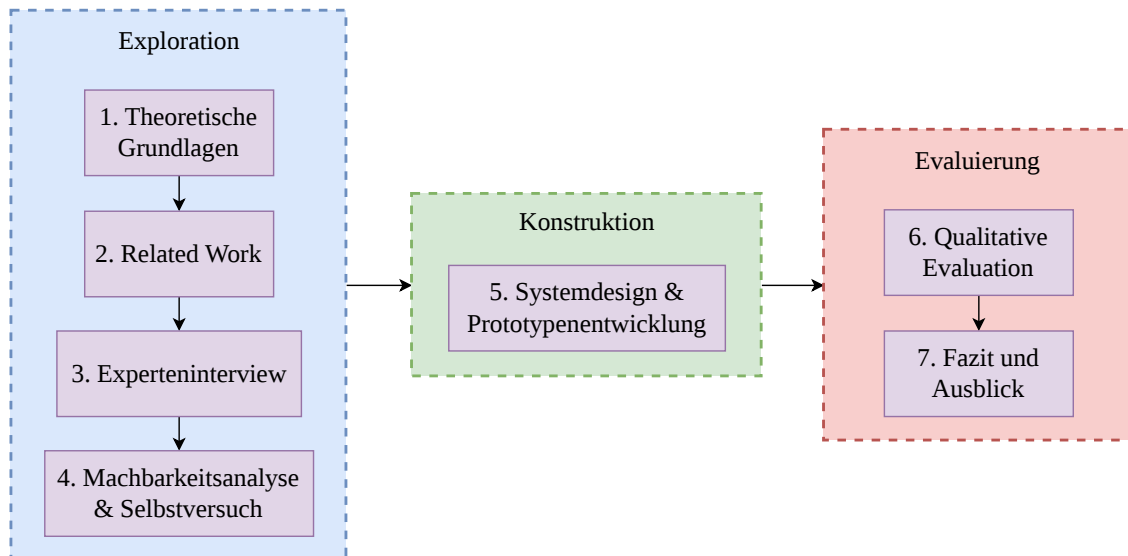


Abbildung 1.2: Aufbau der Arbeit in Anlehnung an die Phasen des Design Science Research. (Eigene Darstellung)

durch. Beschreibt im zweiten Teil die methodische Dekonstruktion auf Code-Ebene, aus der die Aufgabenverteilung für die Agenten-Rollen abgeleitet wird.

Kapitel 6 – Systemdesign und Prototypenentwicklung: Dokumentiert die iterative Konstruktion der Architektur des Prototypen

Kapitel 7 – Qualitative Evaluation: Umfasst die Überprüfung des entwickelten Artefakts durch einen Fachexperten, um den funktionalen Mehrwert und die Limitierungen aufzuzeigen.

Kapitel 8 – Fazit und Ausblick: Schließt die Arbeit mit der Beantwortung der Forschungsfragen, einer Zusammenfassung und einem Ausblick auf zukünftige Forschungspotenziale ab.

Kapitel 2

Theoretische Grundlagen

Dieses Kapitel bildet das theoretische Fundament der Arbeit. Es erklärt die wichtigsten Konzepte der SAP-Variantenkonfiguration, der generativen künstlichen Intelligenz (KI) und der Orchestrierung von Multi-Agenten-Systemen.

2.1 Domänenspezifische Grundlagen: SAP Variantenkonfiguration

2.1.1 Einführung in die logische Architektur der Variantenkonfiguration

Die Idee der Variantenkonfiguration ist es, ein anpassbares Produkt (wie ein Auto) im SAP-System nicht als unzählige separate Endprodukte anzulegen. Stattdessen gibt es ein zentrales Grundmodell, das durch verschiedene Merkmale genauer bestimmt wird. Ein logisches Regelwerk verknüpft diese Merkmale und beachtet dabei technische und kaufmännische Einschränkungen. Weil die Kombination aller Merkmale schnell zu Millionen oder gar Quadrillionen (10^{24}) Varianten führen kann, ist es in der Praxis unmöglich, für jede Ausprägung eigene Stammdaten anzulegen (Blumöhr et al., 2023, S. 86).

Um diese Variantenvielfalt im SAP-System zu bewältigen, gibt es den sogenannten *konfigurierbare Materialstamm* (KMAT). Er dient als Platzhalter, der alle Varianten eines Produkts bündelt (Blumöhr et al., 2023, S. 90). Damit Nutzer das Produkt konfigurieren können, nutzt SAP drei zentrale Bausteine:

- **Merkmale:** Sie definieren die Eigenschaften eines Produkts (z.B. die Lackierung, Motor oder Reifen) (Blumöhr et al., 2023, S. 90 - 96).
- **Variantenklassen:** Sie bündeln die Merkmale und sind direkt dem KMAT zugeordnet (Blumöhr et al., 2023, S. 90 - 96).

- **Konfigurationsprofil:** Es steuert den Konfigurationsprozess und hält die Stammdaten zusammen (Blumöhr et al., 2023, S. 90 - 96).

Ein wichtiges Konzept, um diese Komplexität zu meistern, ist das Maximal-Prinzip. Statt ein Produkt Bauteil für Bauteil zusammenzusetzen (ein additives 100 %-Modell), nutzt SAP ein subtraktives 150 %-Modell. Die Basis dafür ist die *konfigurierbare Maximalstückliste* (Super BOM). Diese Stückliste enthält alle Bauteile, die für jede noch so seltene Variante des Produkts gebraucht werden könnten (Blumöhr et al., 2023, S. 169). Bei der Konfiguration kommen keine neuen Teile dazu – es werden lediglich die nicht benötigten Bauteile herausgefiltert.

Das *Beziehungswissen* verbindet die kaufmännische Konfiguration mit der technischen Fertigung. Es übersetzt den Kundenwunsch in eine konkrete Fertigungsstückliste (Blumöhr et al., 2023, S. 101). Gesteuert wird das beispielsweise durch *Auswahlbedingungen*. Wählt ein Kunde aus sechs möglichen Motoren einen aus, sorgen die Auswahlbedingungen dafür, dass die restlichen fünf Motoren aus der fertigen Stückliste gestrichen werden.

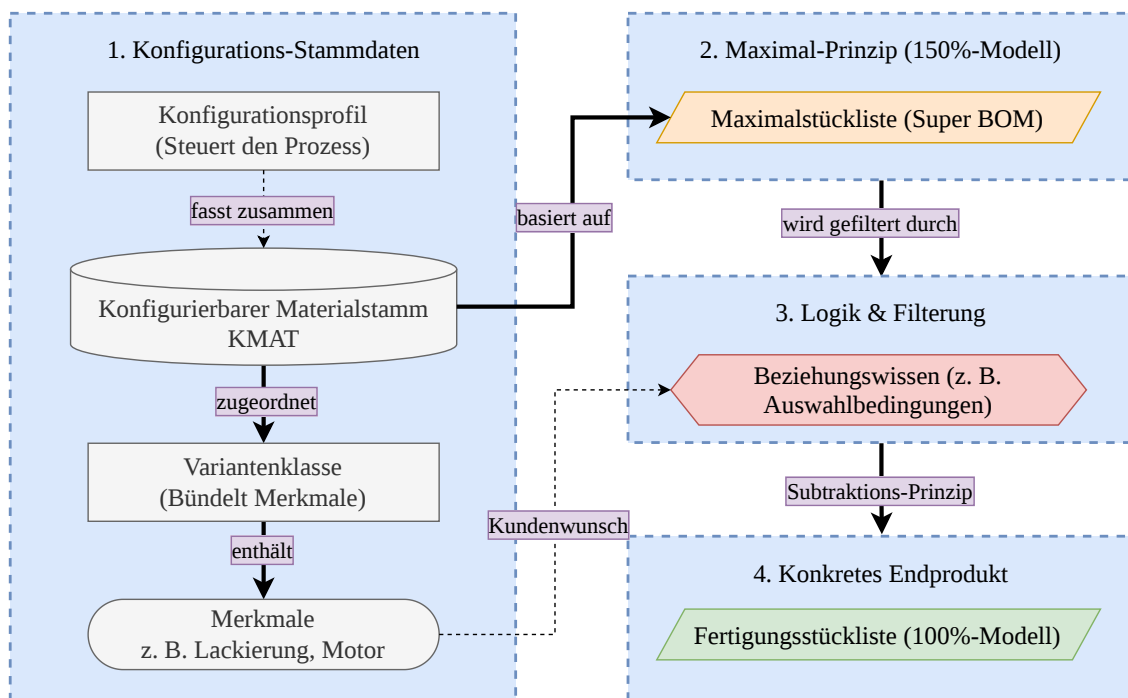


Abbildung 2.1: Logische Architektur der SAP-Variantenkonfiguration (in Anlehnung an Blumöhr et al., 2023).

2.1.2 Die klassische Variantenkonfiguration (LO-VC) im Kontext der Systemtransformation

Um die technologische Entwicklung der Variantenkonfiguration zu verstehen, muss man sich die Evolution der SAP-Systeme ansehen. Während *SAP ERP* (SAP R/3) lange

der Standard war, brachte *SAP S/4HANA* durch die In-Memory-Datenbanktechnologie einen grundlegenden Wechsel (Blumöhr et al., 2023, S. 28). Dadurch wurde auch die Variantenkonfiguration auf ein neues Fundament gestellt.

Der Begriff LO-VC wird heute meist genutzt, um die klassische Konfiguration von der neuen AVC zu unterscheiden (Blumöhr et al., 2023, S. 32). LO-VC arbeitet mit prozeduraler Logik. Weil diese in vielen Unternehmen über Jahre hinweg gewachsen ist, sind die Modelle oft unnötig komplex (Blumöhr et al., 2023, S. 58). Der Wechsel zu AVC ist daher nicht nur ein technisches Update, sondern aus folgenden Gründen eine strategische Notwendigkeit:

- **Vereinfachung:** AVC ist der neue Standard. Die Migration bietet die Chance, historisch gewachsene und schwer wartbare Logiken aufzuräumen (Blumöhr et al., 2023, S. 57).
- **Neue Geschäftsmodelle:** LO-VC ist primär auf den reinen Produktverkauf ausgelegt. Moderne Ansätze wie *Solution Bundling* (die Kombination aus Produkt, Service und Abo) lassen sich im Standard nur mit AVC abbilden (Blumöhr et al., 2023, S. 59).
- **Attraktivität für Fachkräfte:** Die Arbeit mit über 25 Jahre alter Software ist für junge Talente wenig reizvoll. Moderne Architekturen helfen dabei, Fachkräfte zu gewinnen (Blumöhr et al., 2023, S. 60).
- **Bessere Technologie:** AVC nutzt mit *Gecode* einen modernen Constraint-Solver. Im Gegensatz zur schrittweisen LO-VC-Logik arbeitet dieser analytisch, was Leistung und Flexibilität deutlich erhöht (Blumöhr et al., 2023, S. 59).
- **Zukunftssicherheit:** Neue Technologien wie KI oder *Machine Learning* (ML) werden nur noch für AVC entwickelt. Eine nachträgliche Integration in LO-VC wäre zu aufwändig und unzureichend (Blumöhr et al., 2023, S. 61).

Zusammenfassend lässt sich sagen: Wer bei LO-VC bleibt, spart zwar kurzfristig den Aufwand für eine Migration. Langfristig steigt jedoch das Risiko, technologisch den Anschluss zu verlieren und die spätere Umstellung noch teurer zu machen (Blumöhr et al., 2023, S. 61).

2.1.3 Die moderne Architektur (AVC) und das Constraint Paradigma

Die Antwort auf die Schwächen der klassischen Variantenkonfiguration ist die AVC in SAP S/4HANA. Um den wachsenden Anforderungen an Leistung und komplexe Modelle gerecht

zu werden, hat SAP nicht nur die Oberfläche modernisiert, sondern die zugrundeliegende Rechenlogik komplett ausgetauscht.

Während in LO-VC Regeln Schritt für Schritt abarbeitet, nutzt AVC einen deklarativen Ansatz. Das Herzstück der AVC ist der *Constraint Solver* namens *Gecode*. Diese quelloffene Softwarebibliothek wurde zusammen mit dem Fraunhofer ITWM tief in SAP integriert und für Produktkonfigurationen optimiert (Blumöhr et al., 2023, S. 35).

Durch diese Engine wird die Konfiguration zu einem mathematischen *Constraint Satisfaction Problem* (CSP). In der Informatik ist ein CSP ein System aus Variablen, Wertebereichen (Domänen) und Bedingungen (Constraints), die festlegen, welche Kombinationen zulässig sind (Russell, 2010, S. 180).

Dieser Wechsel hat große Auswirkungen für die Nutzer: Sobald jemand in der Konfiguration einen Wert wählt, schränkt die Gecode-Engine in Echtzeit die Auswahlmöglichkeiten aller anderen abhängigen Merkmale ein. Statt also einen Fehler erst am Ende einer Berechnung auszugeben, blendet das System unmögliche Optionen im Vorfeld aus. Das verhindert fehlerhafte Konfigurationen (Blumöhr et al., 2023, S. 35).

Außerdem bietet AVC bessere Analysewerkzeuge wie den *Dependency Trace*. Damit können Entwickler genau sehen, welche Regel dafür gesorgt hat, dass ein bestimmtes Merkmal eingeschränkt wurde (Blumöhr et al., 2023, S. 37).

Kurz gesagt ändert AVC die Herangehensweise: Statt zu fragen „Wie berechne ich die Variante Schritt für Schritt?“, fragt das System „Welche Bedingungen müssen für das Endprodukt erfüllt sein?“. Dieser Wechsel von einer schrittweisen zu einer netzwerkbasierter Logik ist eine Hürde für Unternehmen, wenn sie alte Modelle in das neue System übertragen wollen.

2.1.4 Strukturelle Härte der Migration: Vorbedingungen vs. Constraint-Tabellen

Um nachvollziehen zu können, warum die automatisierte Migration von LO-VC nach AVC ein Problem darstellt, muss der Paradigmenwechsel auf syntaktischer Ebene betrachtet werden.

In der klassischen LO-VC-Welt wird die Konfigurationslogik häufig über isolierte *Vorbedingungen* (Preconditions) modelliert. Diese Vorbedingungen sind imperativ und meist direkt an einzelne Ausprägungen eines Merkmals geheftet. Ein Entwickler definiert beispielsweise direkt an dem Wert 'Ledersitze', dass dieser nur wählbar ist, wenn `Modell = 'Premium' AND Land = 'DE'` gilt. Bei historisch gewachsenen Modellen führt dies zu hunderten, dezentral verstreuten Wenn-Dann-Bedingungen, die isoliert voneinander abgearbeitet werden.

In der neuen AVC-Architektur ist diese fragmentierte Logik ineffizient. Stattdessen nutzt der *Gecode*-Solver eine Zentralisierung der Logik in mathematisch strikten Matrizen,

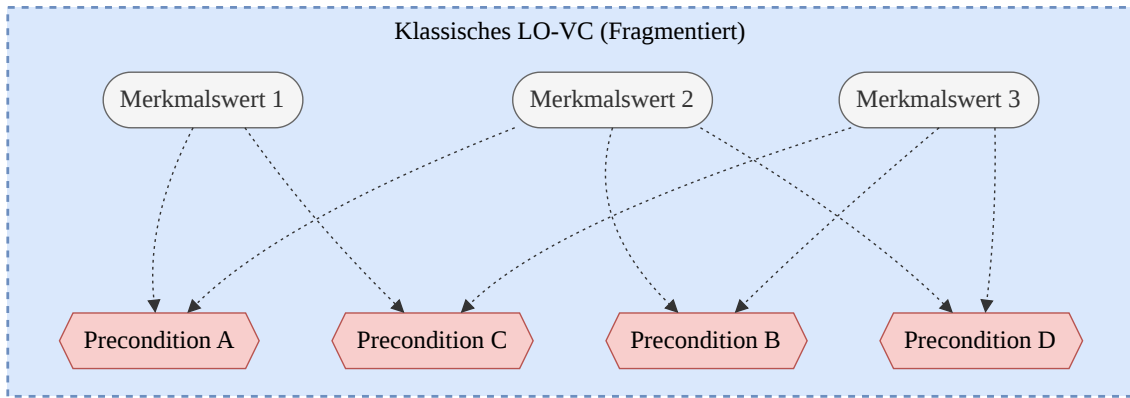


Abbildung 2.2: Bedingungen in der klassischen Variantenkonfiguration (Eigene Darstellung)

den sogenannten *Variantentabellen*. Anstatt Werte einzeln abzufragen, wird eine Tabelle aufgespannt, deren Spalten die interagierenden Merkmale repräsentieren (z.,B. **Modell**, **Land**, **Sitzbezug**). Ein AVC-Constraint ruft diese Tabelle anschließend über eine restriktive Syntax wie `RESTRICT TABLE (...)` auf. Das System gleicht dann die Eingaben des Nutzers global mit den gültigen Zeilenkombinationen der Tabelle ab.

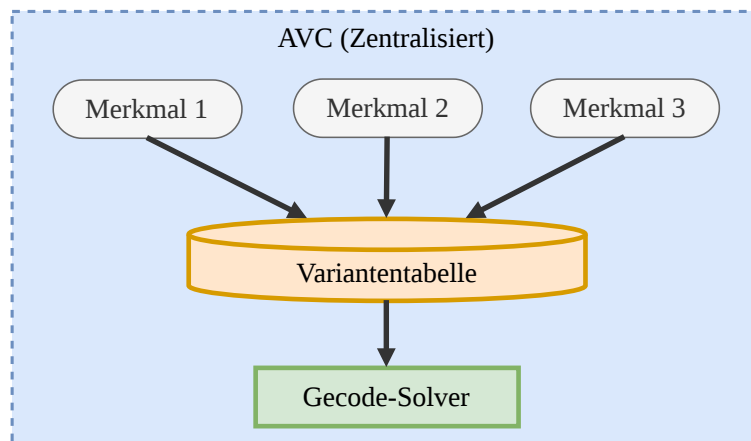


Abbildung 2.3: Bedingungen in der Advanced Variant Configuration (Eigene Darstellung)

Die Migration von LO-VC nach AVC ist an dieser Stelle keine reine linguistische Code-Übersetzung, sondern eine fundamentale logische Restrukturierung, die keinen Raum für syntaktische Unschärfen lässt:

- **Aggregation von Logik:** Verstreute Einzelbedingungen müssen algorithmisch erkannt und nach identischen Merkmalskombinationen in einer gemeinsamen Tabellenstruktur gebündelt werden.
- **Lückenlose Matrix (Wildcards):** Im Gegensatz zu isolierten LO-VC-Skripten dulden AVC-Tabellen keine logischen Lücken. Werte, die in der alten Welt an keine

Bedingung geknüpft waren (und somit immer gültig sind), müssen in der neuen Matrix zwingend durch Platzhalter (Wildcards) definiert werden. Geschieht dies nicht, bildet der Constraint-Solver fehlerhafte Schnittmengen und sperrt gültige Konfigurationen aus.

Genau diese strukturelle Härte und die Notwendigkeit, aus unscharfem historischen Code eine deterministische und fehlerfreie Tabellen-Matrix abzuleiten, bilden den Kern des in dieser Arbeit untersuchten Problemraums.

2.2 Generative künstliche Intelligenz im Software Engineering

2.2.1 Large Language Models für Quellcode

In den letzten Jahren haben KI-Systeme enorme Fortschritte gemacht. Im Zentrum stehen dabei *Large Language Models* (LLMs). Das sind große neuronale Netze, die mit riesigen Textmengen trainiert wurden, um das wahrscheinlichste nächste Zeichen in einem Text vorherzusagen. Was als einfaches Werkzeug zur Textvervollständigung begann, hat sich zu leistungsstarken Systemen entwickelt, die auch im Software Engineering eingesetzt werden. Um diese Modelle für die automatisierte Code-Migration nutzen zu können, muss man verstehen, wie sie funktionieren und wie man sie steuert.

Die Transformer-Architektur und der Aufmerksamkeitsmechanismus

Die meisten heutigen Sprachmodelle basieren auf der Transformer-Architektur, die von Vaswani et al. (2017) vorgestellt wurde. Der große Durchbruch dieser Architektur ist, dass Texte oder Quellcode nicht mehr strikt nacheinander (Wort für Wort) verarbeitet werden müssen, wie es bei älteren Modellen der Fall war.

Der sogenannte *Self-Attention-Mechanismus* ermöglicht es, alle Wörter eines Textes gleichzeitig zu verarbeiten. Das Modell berechnet für jedes Wort, wie wichtig es im Verhältnis zu allen anderen Wörtern ist.

Für das Programmieren ist das besonders nützlich, weil es das Problem der weitreichenden Abhängigkeiten löst. Wenn eine Variable ganz oben im Code definiert und hunderte Zeilen später genutzt wird, erkennt das Modell diese Verbindung sofort – unabhängig davon, wie weit die beiden Stellen auseinanderliegen. Der Quellcode wird vom Modell also als ein Netz zusammenhängender Logikbausteine verstanden.

In-Context Learning und Few-Shot Prompting

Früher musste man neuronale Netze für neue Aufgaben aufwändig nachtrainieren (*Fine-Tuning*). Moderne LLMs sind jedoch ab einer gewissen Größe so leistungsstark, dass das oft nicht mehr nötig ist.

Brown et al. (2020) zeigten, dass große Modelle Aufgaben direkt durch die Eingabe (den Prompt) erlernen können. Das nennt man *In-Context Learning* (ICL). Das Modell nutzt sein allgemeines Wissen, um eine neue Aufgabe anhand der Beschreibung im Prompt zu lösen.

Eine sehr effektive Methode dafür ist das *Few-Shot Prompting*. Dabei gibt man dem Modell neben der eigentlichen Aufgabe ein paar konkrete Lösungsbeispiele mit. Das Modell erkennt die Logik in den Beispielen und wendet sie auf das neue Problem an, ohne dass sein zugrundeliegendes Netzwerk umprogrammiert werden muss.

Das Paradoxon der Code-Übersetzung

Wenn es darum geht, welches Modell man für Software-Projekte nutzt, liefert Zheng et al. (2023) eine wichtige Erkenntnis: Es gibt nicht das eine perfekte Modell für alles.

Einerseits gibt es kleinere, auf Programmierung spezialisierte Modelle (*Code LLMs*). Wenn es darum geht, neuen Code zu schreiben oder Standardaufgaben (wie Unit-Tests) zu lösen, sind diese Modelle oft besser und effizienter als riesige Universalmodelle.

Geht es jedoch darum, bestehenden Code in eine andere Sprache oder Logik zu übersetzen (*Code Translation*), sind die großen, allgemeinen Modelle (wie GPT-4) im Vorteil.

Der Grund dafür liegt in der Aufgabe selbst: Code-Migration ist mehr als nur das Austauschen von Befehlen. Das Modell muss die Absicht und die Logik des alten Codes verstehen, das Konzept abstrahieren und es in die Regeln der neuen Sprache übersetzen. Für diese anspruchsvolle Denkleistung (*Reasoning*) brauchen die Modelle ein breites Wissen und hohe Kapazität. Reine Code-Modelle scheitern oft an diesem nötigen Paradigmenwechsel.

2.2.2 Limitationen monolithischer Ansätze

Auch wenn Sprachmodelle Code syntaktisch gut verstehen und übersetzen können, stößt ihr Einsatz bei komplexen Systemmigrationen an harte Grenzen. Besonders fehleranfällig wird es, wenn versucht wird, komplexe Übersetzungsaufgaben mit einer einzigen Eingabeaufforderung (*Single-Prompt*) in einem Rutsch zu lösen. Solche isolierten Versuche nennt man *monolithische Ansätze*. Dass diese in der Praxis scheitern, lässt sich wissenschaftlich auf zwei Hauptursachen zurückführen: die algorithmische Komplexität bei der Generierung und das Scheitern an systemweiten Abhängigkeiten.

Die Komplexitätsfalle und das probabilistische Paradigma

Dass LLMs bei langen, sequenziellen Logikketten schwächeln, zeigten Chen et al. (2021) bereits bei der Evaluation früherer Code-Modelle wie Codex. Sie bewiesen, dass Modelle kurze, isolierte Aufgaben zwar zuverlässig lösen können, die Leistung jedoch drastisch einbricht, sobald eine Aufgabe viele aneinandergereihte Arbeitsschritte erfordert. Für die automatisierte Migration von SAP-Modellen ist das ein zentrales Problem: Wenn ein Modell gezwungen wird, eine komplett neue Constraint-Architektur mit all ihren Tabellen und Vorbedingungen auf einmal zu entwerfen, überfordert das die kognitive Kapazität (das *Reasoning*) des **LLM!** (**LLM!**). Das Modell geht dann in ein rein probabilistisches Paradigma über – es generiert den Code nur noch basierend auf Wahrscheinlichkeiten, ohne die mathematische Logik im Hintergrund tiefgreifend zu prüfen. Das Paper beschreibt zudem, dass die Wahrscheinlichkeit für einen direkten Treffer im ersten Versuch (Metrik *pass@1*) bei solch komplexen Aufgaben extrem gering ist. Erst wenn ein System in der Lage ist, iterativ Lösungswege zu generieren, diese zu prüfen und zu verfeinern, steigt die Erfolgsquote. Ein monolithischer Ansatz bietet jedoch keinerlei Mechanismus für solche kognitiven Prüfschleifen.

Das Scheitern an systemweiten Abhängigkeiten und fehlendem Umgebungs-Feedback

Neben der sequenziellen Komplexität stellt die globale Vernetzung von Code eine weitere Hürde dar. Jimenez et al. (2024) untersuchten mit dem *SWE-bench*-Benchmark reale Software-Projekte und zeigten auf, dass LLMs massive Probleme haben, wenn Code-Bausteine systemweite Folgen nach sich ziehen. Selbst Top-Modelle wie GPT-4 scheiterten im monolithischen Ansatz und konnten kaum 5% der Probleme lösen. Obwohl bei der Systemtransformation ein völlig neues AVC-Zielmodell generiert wird, lassen sich diese Erkenntnisse direkt auf die **SAP!** (**SAP!**)-Variantenkonfiguration übertragen:

1. **Fehlender Überblick über Netzwerke:** Selbst bei der Generierung eines komplett neuen Modells verliert ein monolithisches LLM schnell den Überblick über die strikten, netzwerkartigen Abhängigkeiten der deklarativen Constraint-Architektur. Das Modell generiert beispielsweise isolierte Tabellen für einzelne Merkmale, berücksichtigt dabei aber nicht die globalen Schnittmengen und Wechselwirkungen mit anderen, parallel generierten Constraints. Bei der vernetzten AVC-Architektur führt das unweigerlich zu fehlerhafter Logik.
2. **Fehlendes Feedback (Blindflug):** Ein monolithischer Ansatz operiert im luftleeren Raum. Das Modell schreibt Code, kann ihn aber nicht autonom verifizieren. In der realen Softwareentwicklung ist Programmieren jedoch ein iterativer Prozess aus Schreiben und Testen. Wird ein Sprachmodell lediglich über einen isolierten

Prompt gesteuert und ist nicht architektonisch in eine dynamische Feedback-Schleife (beispielsweise mit einer Test- oder Compilerumgebung) eingebunden, muss es blind raten, ob die erzeugte Struktur funktionsfähig ist.

Zusammenfassend lässt sich festhalten, dass einfache KI-Prompts für den Aufbau komplexer, voneinander abhängiger Architektur-Modelle ungeeignet sind. Um die Fehleranfälligkeit des fehlenden Feedbacks und der Komplexitätsfalle abzufangen, bedarf es strukturierter Multi-Agenten-Systeme, die Aufgaben in analytische Zwischenschritte dekonstruieren.

2.3 Multi-Agenten-Systeme (MAS)

2.3.1 Definition und kognitive Architekturen

Um die eben beschriebenen Schwächen einzelner Sprachmodelle auszugleichen, setzt die Informatik zunehmend auf MAS. Die Idee ist simpel: Statt eine schwere Aufgabe in einen einzigen Prompt zu packen, verteilt man sie auf mehrere spezialisierte KI-Agenten, die zusammen auf ein Ziel hinarbeiten.

Der Begriff des LLM-basierten Agenten

Klassisch wird ein Agent als System definiert, das seine Umwelt wahrnimmt und darauf reagiert (Russell, 2010, S. 36). Bei generativer KI dient das Sprachmodell (LLM) als "Gehirn" des Agenten.

Ein solcher KI-Agent ist mehr als ein normaler Chatbot. Er arbeitet selbstständig, verfolgt Ziele und kann Werkzeuge nutzen. Um einen Agenten zu erstellen, gibt man dem Sprachmodell klare Anweisungen zu seiner Rolle und seinen Aufgaben. Zudem erhält der Agent Zugang zu externen Tools oder Datenbanken.

Ein großer Vorteil von Multi-Agenten-Systemen ist ihre Flexibilität. Es gibt keine starren Regeln für ihren Aufbau. Man unterscheidet grob zwei Ansätze:

1. **Statische Orchestrierung:** Der Entwickler gibt genaue Ablaufpläne vor (z.,B. Agent A übergibt sein Ergebnis immer an Agent B).
2. **Dynamische Autonomie:** Das System entscheidet selbstständig, wann welcher Agent oder welches Werkzeug gebraucht wird, um ein Teilproblem zu lösen.

Die kognitive Kern-Anatomie nach Wang et al.

Auch wenn die Umsetzung variieren kann, besteht ein leistungsfähiger Agent laut Wang et al. (2024) meist aus vier zentralen Bausteinen:

- **Profil (Rolle):** Hier wird festgelegt, welche Expertise der Agent hat (z.,B. „SAP-Entwickler“). Das hilft dem Modell, sein Wissen gezielt auf diese Rolle zu fokussieren.

- **Gedächtnis:** Ein Kurzzeitgedächtnis merkt sich den aktuellen Verlauf der Unterhaltung, ein Langzeitgedächtnis speichert gelernte Regeln oder gelöste Probleme dauerhaft.
- **Planung:** Der Agent zerlegt ein großes Ziel in machbare Teilaufgaben und setzt Prioritäten.
- **Aktion (Werkzeuge):** Der Agent setzt seine Pläne in echte Handlungen um, indem er Datenbanken abfragt oder Code ausführt.

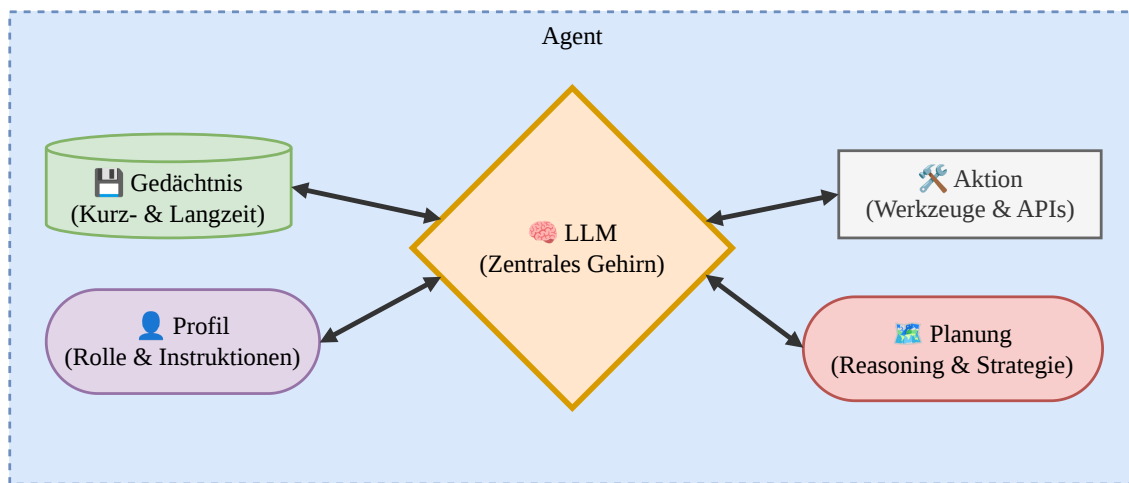


Abbildung 2.4: Zentrale Architektur eines autonomen LLM-Agenten (in Anlehnung an Wang et al., 2024).

Das ReAct-Paradigma zur dynamischen Fehlerkorrektur

Damit sich Fehler nicht unbemerkt durch das System ziehen, nutzen moderne Agenten das *ReAct*-Prinzip (Reasoning and Acting) von Yao et al. (2022).

Dabei arbeitet der Agent in einer ständigen Schleife aus drei Schritten:

Thought (Gedanke) $\longrightarrow$ Action (Handlung) $\longrightarrow$ Observation (Beobachtung)

Zuerst analysiert der Agent das Problem (Thought) und entscheidet sich für eine Handlung. Dann führt er diese Handlung mit einem Werkzeug aus (Action). Das entscheidende Element ist die Beobachtung (Observation): Das Ergebnis der Handlung (z.,B. eine Fehlermeldung) wird als neues Wissen an den Agenten zurückgespielt.

So kann der Agent Fehler erkennen und seinen Code im nächsten Versuch anpassen. Dieses selbstkorrigierende Verhalten verhindert, dass das Modell einfach falschen Code produziert, und ist für komplexe SAP-Migrationen essenziell.

2.3.2 Kollaborative Agenten-Muster

Dieser Abschnitt beleuchtet, wie mehrere KI-Agenten zusammenarbeiten können. Die Forschung hat verschiedene Muster entwickelt, um Aufgaben effizient zu verteilen und Fehler zu minimieren.

Rollenspezialisierung und Dehalluzination

Eine sehr gute Methode ist es, Agenten klare Expertenrollen zuzuweisen (z.,B. Programmierer, Architekt, Tester). Das *ChatDev*-Framework von Qian et al. (2024) zeigt, dass dies isolierten Ansätzen weit überlegen ist. Wichtig ist dabei die sogenannte kommunikative Dehalluzination: Die Agenten sollen nicht einfach raten, wenn Informationen fehlen, sondern beim zuständigen Agenten nachfragen und Details einfordern.

Inception Prompting und System-Leitplanken

Damit die Agenten ihre Rollen nicht verlassen, brauchen sie feste Regeln. Li et al. (2023) nutzen im *CAMEL*-Framework klare Systemanweisungen und Kommunikationsprotokolle, um Endlosschleifen oder ungewollte Rollenwechsel zu verhindern.

Werkzeugnutzung und Code-Ausführung

Agenten sprechen nicht nur miteinander, sondern nutzen auch Werkzeuge. Das *AutoGen*-Framework von Wu et al. (2024) setzt dafür spezielle PProxy-Agenten ein. Deren einzige Aufgabe ist es, den generierten Code auszuführen und mögliche Fehlermeldungen direkt an die anderen Agenten zurückzumelden. So wird der Code direkt auf seine Funktionalität geprüft.

Iterative Fehlerkorrektur (Self-Refinement)

Um auf Fehler gut reagieren zu können, müssen die Agenten strukturiert nachdenken. Shinn et al. (2023) beschreiben im *Reflexion*-Framework, wie Agenten aus Fehlern lernen: Statt einfach einen neuen Code zu raten, analysiert der Agent den Fehler, entwickelt eine Lösungsstrategie, speichert diese ab und passt den Code gezielt an.

Das Fließband-Paradigma

Hong et al. (2024) fassen all diese Muster im *MetaGPT*-Framework zusammen. Sie lassen die Agenten wie am Fließband arbeiten und strikte Arbeitsanweisungen (SOPs) befolgen. Die Agenten chatten dabei nicht frei miteinander, sondern tauschen klar strukturierte Dokumente aus. Die Studie zeigt, dass diese strenge Organisation herkömmliche Ansätze deutlich übertrifft.

2.3.3 Entwicklungs-Frameworks für MAS

Um Agenten-Systeme zu programmieren, greift man meist auf fertige Frameworks zurück. Diese Bibliotheken übernehmen Standardaufgaben wie die Verwaltung von Schnittstellen oder des Gedächtnisses.

Eine Studie von Liu et al. (2025) zeigt, dass vor allem drei Frameworks den Markt dominieren: *LangChain*, *AutoGen* und *CrewAI*. Sie unterscheiden sich grundlegend:

- **LangGraph:** Diese Erweiterung von *LangChain* betrachtet Agenten und Werkzeuge als Knoten in einem Graphen. Die Verbindungen dazwischen steuern den Ablauf (Chase, 2024). Das bietet maximale Flexibilität, erfordert aber viel Programmierarbeit.
- **AutoGen:** Das Framework von Microsoft setzt auf Dialoge. Die Agenten lösen Aufgaben, indem sie miteinander chatten. Ein großer Vorteil ist die einfache Integration von Proxy-Agenten, um Code direkt auszuführen (Wu et al., 2024).
- **CrewAI:** Dieses Framework arbeitet nach dem Fließband-Prinzip (ähnlich wie MetaGPT). Ein System steuert verschiedene Agenten, die klare, aufeinanderfolgende Aufgaben abarbeiten. Der Fokus liegt auf festen Rollen und Aufgabenverteilungen (Liu et al., 2025).

Framework	Architektur-Paradigma	Stärken	Schwächen
LangChain (LangGraph)	Graphbasiert (Zustandsautomaten)	Höchste Kontrolle über den Ablauf; gigantisches Ökosystem an Integrationen.	Sehr steile Lernkurve; viel Boilerplate-Code für einfache Multi-Agenten-Szenarien.
AutoGen	Chatbasiert (Conversational)	Out-of-the-Box Multi-Agenten-Dialoge; starke native Code-Ausführung.	Gefahr von Endlosschleifen bei unklarer Steuerung; Kommunikationsfluss teils unvorhersehbar.
CrewAI	SOP / Fließband (Rollen & Tasks)	Extrem entwicklerfreundlich; klare Delegation und strukturierte Prozesse.	Weniger flexibel bei hochdynamischen oder stark verzweigten Problemstellungen.

Tabelle 2.1: Vergleich der dominierenden Multi-Agenten-Frameworks (in Anlehnung an Liu et al., 2025).

Mapping von Theorie und Framework-Praxis

Diese Frameworks nehmen Entwicklern viel Routinearbeit ab (z.,B. durch integrierte Datenbanken für das Gedächtnis oder vorbereitete ReAct-Schleifen). Sie kennen jedoch keine domänenspezifischen Themen wie SAP-Architekturen. Für den Entwickler bleiben deshalb zwei Hauptaufgaben:

1. **Systemdesign:** Man muss entscheiden, welches der Agenten-Muster am besten zur Aufgabe passt. Anschließend muss man dem Framework die Rollen, Arbeitsanweisungen und Regeln einprogrammieren.
2. **Werkzeug-Entwicklung:** Es gibt keine fertigen Werkzeuge für SAP-Migrationen. Die Brücke zwischen den Agenten und der Zielumgebung (etwa ein Compiler zur Überprüfung des SAP-Codes) muss komplett selbst entwickelt werden.

Mit den hier erläuterten Konzepten – von den SAP-Konfigurationsmodellen über die Schwächen einzelner KI-Modelle bis hin zu den Architekturen moderner Agenten-Frameworks – sind die technologischen Grundlagen für diese Masterarbeit geschaffen. Diese Erkenntnisse bilden das theoretische Fundament, auf dem die nachfolgenden Entscheidungen für die eigene Migrationsarchitektur aufbauen.

Kapitel 3

Related Work

Die Automatisierung komplexer Software-Migrationen erfordert ein tiefgreifendes Verständnis sowohl der proprietären Zielsysteme (hier SAP) als auch aktueller KI-Technologien. Da die direkte Verbindung von SAP-Migrationen und generativer KI bislang wenig erforscht ist, ordnet dieses Kapitel die vorliegende Arbeit anhand aktueller Publikationen aus verwandten Disziplinen in den Stand der Technik ein. Zunächst analysiert das Kapitel die konkreten Hürden bei der Transformation von SAP-Konfigurationsmodellen und beleuchtet die Limitierungen aktueller Standardwerkzeuge. Darauf aufbauend wird der Einsatz von Large Language Models (LLMs) für die Code-Migration betrachtet. Dabei wird aufgezeigt, warum monolithische Ansätze bei strengen Architekturvorgaben in der Praxis häufig scheitern. Abschließend stellt das Kapitel den aktuellen Forschungsstand zu Multi-Agenten-Systemen (MAS) vor und leitet daraus die zentrale Forschungslücke dieser Arbeit ab: die semantische Migration proprietärer SAP-Constraint-Logik.

Bei der Migration auf SAP S/4HANA stehen Unternehmen vor der Aufgabe, ihre historisch gewachsenen Produktmodelle von der klassischen Variantenkonfiguration (LO-VC) in die Advanced Variant Configuration (AVC) überführen. Matilainen (2024, S. 31) zeigt in seiner Studie die Komplexität dieses Wechsels auf: Da die Zahl der möglichen Konfigurationen exponentiell mit den Merkmalen und Optionen wächst, ist eine präzise Restrukturierung der Datenmodelle erforderlich, um die Variantenvielfalt im neuen System technisch beherrschen zu können.

Die Entwicklungen im Bereich der Large Language Models (LLMs) haben die Möglichkeiten zur automatisierten Code-Generierung und -Transformation grundlegend erweitert (Karabiyik, 2025). Die aktuelle Forschung belegt jedoch, dass die Migration komplexer Altsysteme nicht durch eine einzige LLM-Abfrage (Single-Prompt oder Monolithic Prompting) bewältigt werden kann. Derartige Ansätze scheitern in der Praxis häufig an der unzureichenden Berücksichtigung spezifischer Architekturvorgaben und systeminterner Abhängigkeiten. Moti et al. (2026) demonstrieren am Beispiel des Frameworks Legacy-Translate, dass naive LLM-Übersetzungen bei industriellen Codebasen Kompilierraten von 0 % aufweisen können. Obwohl der generierte Code syntaktisch plausibel erscheint,

ist er aufgrund fehlender interner Schnittstellen (APIs) und domänenspezifischer Logik nicht funktionsfähig (Moti et al., 2026). Ähnliche Defizite dokumentieren Xu et al. (2025): Während spezialisierte Systeme bei strukturellen Code-Änderungen hohe Erfolgsquoten erzielen, erreichen monolithische Basismodelle lediglich eine Erfolgsquote von 8,7 %.

Ein weiteres konzeptionelles Problem dieser One-Shot-Ansätze liegt in ihrer mangelnden Kontrollierbarkeit (Karabiyik, 2025; Oueslati et al., 2026). Ohne eine Modularisierung in kleinere Arbeitsschritte bleibt der Transformationsprozess eine intransparente Blackbox. Es können keine funktionalen Garantien für den Zielcode abgegeben werden, und die Modelle neigen bei komplexeren Interaktionen zu Halluzinationen (Karabiyik, 2025; Oueslati et al., 2026). Um diese Unzulänglichkeiten zu überwinden, fokussiert sich die aktuelle Forschung zunehmend auf Multi-Agenten-Systeme (MAS). Deren Grundprinzip besteht darin, einen komplexen Prozess in logische Teilaufgaben zu zerlegen, die anschließend von kooperierenden, spezialisierten KI-Agenten autonom bearbeitet werden.

Erste Studien im Bereich des automatisierten Software-Refactorings – der strukturellen Optimierung innerhalb derselben Programmiersprache – belegen die prozessuale Überlegenheit dieses modularen Ansatzes. Das Framework RefactorGPT überführt den Refactoring-Prozess durch eine sequentielle Orchestrierung in einen kontrollierbaren Workflow (Karabiyik, 2025). Auch das Framework RefAgent validiert diesen Vorteil: Die Aufteilung in spezialisierte Agenten für Planung, Ausführung und Fehlerbehebung steigert die Kompiliererfolgsrate signifikant und reduziert fehlerhafte Generierungen (Oueslati et al., 2026). Ein zentraler Erfolgsfaktor sind dabei autonome Feedback-Schleifen. So integriert das System MANTRA einen „Repair Agent“, der Compiler-Fehler iterativ korrigiert, wodurch die Erfolgsquote bei komplexen Transformationen auf 82,8 % gesteigert werden konnte (Xu et al., 2025). Obwohl ein solches direktes Compiler-Feedback in der vorliegenden Arbeit aufgrund restriktiver, proprietärer SAP-Systemarchitekturen noch der Future Work vorbehalten bleibt, unterstreichen diese Ergebnisse das technologische Potenzial von MAS.

Während die Refactoring-Ansätze die Effizienz der Agenten-Kollaboration validieren, demonstriert das Projekt LegacyTranslate (Moti et al., 2026), dass sich diese Architektur auch auf deutlich komplexere Cross-Language-Migrationen übertragen lässt. Die Übersetzung von PL/SQL zu Java erfordert hier einen Paradigmenwechsel. LegacyTranslate implementiert hierfür ein „API Grounding“, durch welches das System aktiv an die Architekturvorgaben des Zielsystems gebunden wird.

Trotz der empirisch belegten Leistungsfähigkeit von MAS bei der Code-Migration offenbart die aktuelle Literatur eine signifikante Lücke hinsichtlich proprietärer ERP-Ökosysteme. Etablierte Frameworks wie MANTRA oder LegacyTranslate fokussieren sich primär auf imperative oder objektorientierte Standardsprachen (wie Java oder PL/SQL). Bislang wurde nicht wissenschaftlich untersucht, inwiefern kollaborative KI-Architekturen auf die domänenspezifische Constraint-Logik der SAP angewendet werden können.

Es bleibt folglich eine offene Forschungsfrage, ob Multi-Agenten-Systeme fähig sind, historisch gewachsene proprietäre Logik semantisch zu analysieren und in ein deklaratives Format zu überführen. Für diesen notwendigen Paradigmenwechsel bieten bestehende KI-Ansätze noch keine architektonische Lösung.

An dieser methodischen Schnittstelle setzt die vorliegende Arbeit an. Sie adressiert die identifizierte Forschungslücke, indem sie das Konzept verteilter Agenten-Architekturen auf die komplexe Migration der SAP-Variantenkonfiguration überträgt, um genau diese fehlende analytische und logische Brücke zwischen Alt- und Neusystem zu schlagen.

Kapitel 4

Experteninterview

Dieses Kapitel dokumentiert die Durchführung und Auswertung des qualitativen Experteninterviews. Innerhalb des gestaltungsorientierten Forschungsdesigns verbinden die Erkenntnisse das technische System mit dem organisationalen Umfeld (*Environment*). Sie bilden die empirische Basis, um die manuelle Baseline und die funktionalen Anforderungen an das Multi-Agenten-System (MAS) zu definieren.

4.1 Methodisches Setup und Durchführung

Um die prozessualen Hürden und die zeitliche Baseline der manuellen Migration fundiert zu erfassen, wurde das Forschungsdesign für diese initiale Anforderungsanalyse als explorative Fallstudie gemäß den etablierten Richtlinien für das Software Engineering nach Runeson und Höst strukturiert (Runeson & Höst, 2009). Das methodische Vorgehen definiert sich durch folgende Kernkriterien:

- **Zielsetzung und Forschungsfragen (Objective & Research Questions):**
Kapitel 2.1.2 hat bereits aufgezeigt, wie relevant und kritisch die Migration von LO-VC zu AVC ist. Um ein MAS fundiert zu konzipieren, reicht es jedoch nicht aus, lediglich theoretische Syntaxunterschiede zu analysieren. Historisch gewachsene Praxisprobleme sind stark kontextabhängig und gehen oft nicht aus der reinen Systemdokumentation hervor. Zudem fehlen in der Fachliteratur quantitative Daten zum tatsächlichen manuellen Migrationsaufwand. Ziel dieses Kapitels ist die Beantwortung der ersten Forschungsfrage und damit die Identifikation der prozessualen Hürden bei der manuellen Migration historischer SAP-Modelle sowie die Ermittlung des bisherigen zeitlichen Aufwands zur Definition einer expertenbasierten Baseline.
- **Fallauswahl und Selektionsstrategie (The Case & Selection Strategy):**
Der untersuchte „Fall“ dieser explorativen Datenerhebung ist ein qualitatives Interview, das mit einem erfahrenen Fachexperten (Product Owner und Softwareentwickler) der *msg systems ag* geführt wurde. Hinsichtlich der Selektionsstrategie

handelt es sich um eine pragmatische Auswahl nach Verfügbarkeit. Da der Zugang zu hochspezialisiertem Personal im proprietären SAP-Beratungsumfeld innerhalb der *msg systems ag* stark limitiert ist, fiel die Wahl auf diesen Experten, da er für das Forschungsprojekt zeitlich und organisatorisch zur Verfügung stand. Wie Runeson und Höst (2009) festhalten, ist ein solches Vorgehen in industriellen Fallstudien der Softwaretechnik gängige Praxis. Der Befragte erfüllt die inhaltlichen Anforderungen an einen Experten: Er verfügt über zwölf Jahre Erfahrung als Maschinenbauingenieur und ist seit über sechs Jahren auf die SAP-Variantenkonfiguration spezialisiert. Seit knapp vier Jahren betreut er primär Migrationen von LO-VC nach AVC im msg-Kontext und repräsentiert somit exakt die fachliche Zielgruppe des zu entwickelnden Systems.

- **Datenerhebungsmethode (Methods):** Da die Code-Transformation maßgeblich von undokumentiertem Erfahrungswissen (*Tacit Knowledge*) geprägt ist, erfordert die Durchdringung dieser Problemstellung einen qualitativen Forschungsansatz (Seaman, 1999). Als Erhebungsinstrument diente ein semi-strukturiertes Interview. Der flexible Leitfaden kombinierte offene und spezifische Fragen. Dies stellte sicher, dass antizipierte Daten, wie zeitliche Metriken, systematisch erfasst wurden, ließ aber gleichzeitig Raum für unvorhergesehene Einblicke in die Migrationspraxis (Seaman, 1999). Im Gegensatz zu sozialwissenschaftlichen Interviews erfordert dieser Ansatz im Software Engineering ein tiefgreifendes technisches Vorverständnis des Interviewers, um fachspezifische Nuancen korrekt zu erfassen und zu hinterfragen (Hove & Anda, 2005). Deshalb wurde das Interview bewusst erst nach einer Einarbeitungsphase des Interviewers durchgeführt. Das etwa 45-minütige Gespräch wurde am 17.03.2026 vom Autor dieser Arbeit moderiert und mit Einverständnis des Experten digital aufgezeichnet.
- **Datenauswertung und Codierungsverfahren (Data Analysis):** Zur Wahrung der Datenqualität und wissenschaftlichen Integrität durchlief das Rohmaterial zunächst drei Aufbereitungsschritte: (1) Manuelle Anonymisierung aller personen- und projektspezifischen Daten, (2) KI-gestützte Glättung von rein sprachlichen Redundanzen zur besseren Lesbarkeit sowie (3) eine manuelle zeilenweise Verifikation durch den Autor. Dieses geglättete finale Transkript (siehe Anhang ??) bildete die Basis für die Codierung. Der Codierungsprozess verlief in Anlehnung an Mayring (2014) in zwei Phasen: Einer deduktiven Kategorienanwendung (theoriegeleitete Anwendung vorab definierter Codes, z.,B. manuelle Prozessschritte, Zeitaufwand) sowie einer induktiven Kategorienbildung zur Erfassung unvorhergesehener Praxisprobleme. Um die methodische Qualität der Fallstudie zu sichern, wird so gemäß den Richtlinien von Runeson und Höst (2009) eine lückenlose Beweiskette (*Chain of Evidence*) etabliert,

die über die folgenden Kapitel hinweg wissenschaftlich überprüfbar macht, wie das Systemdesign schrittweise aus den qualitativen Rohdaten abgeleitet wurde.

4.2 Ergebnisse der Befragung

Die folgende Übersicht fasst die zentralen Erkenntnisse der qualitativen Inhaltsanalyse zusammen, gegliedert in thematische Schwerpunkte. Sie beschreiben den empirischen Ist-Zustand des Migrationsprozesses und bilden die Grundlage für die Anforderungsanalyse sowie die Definition der manuellen Baseline.

4.2.1 Kognitive Altlasten und historischer Code

Historisch gewachsene LO-VC-Modelle enthalten oft viele ungenutzte Altlasten. Merkmale und Beziehungswissen existieren im System, ohne im aktuellen Modellierungskontext funktional relevant zu sein. Zusammen mit stetig wachsenden Werteräumen erhöht dies die Komplexität erheblich. Eine unreflektierte 1:1-Konvertierung in die Advanced Variant Configuration (AVC) greift daher in der Praxis zu kurz. Stattdessen müssen Entwickler zunächst zeitaufwendig zwischen aktiver Logik und „totem Code“ unterscheiden, bevor sie die eigentliche Transformation beginnen können. Dies führt zu einer hohen kognitiven Belastung.

4.2.2 Paradigmenwechsel und syntaktisches Umdenken

Die Befragung verdeutlicht, dass die Migration nicht nur syntaktische Anpassungen, sondern einen fundamentalen Paradigmenwechsel erfordert. Moderne Vorgaben wie die SAP „Clean Core“-Strategie verbieten proprietäre Eigenentwicklungen (z.,B. Z-Functions) im AVC. Zudem ändert sich das Systemverhalten massiv – etwa durch den Zwang zu klassifizierten Merkmalen oder die globale Wirkung von Constraints. Eine direkte Übernahme alter Logiken führt daher oft zu falschen Konfigurationsergebnissen oder schlechter Performance. Entwickler müssen die ursprüngliche Intention des Legacy-Codes vollständig verstehen und semantisch mit den neuen Bordmitteln des AVC abbilden.

4.2.3 Fragmentierung manueller Prozessschritte

Trotz Standard-Tools zur Massenanlage von Objekten bleibt die Migration stark fragmentiert und manuell geprägt. Während Automatisierungen Basisdaten anlegen können, muss der Entwickler die komplexe Logik intellektuell durchdringen. Er wertet Vorbedingungen manuell aus, überführt sie in tabellenbasierte Strukturen und prüft diese auf Konsistenz. Diese ständigen Medienbrüche degradieren den Experten zu einer „manuellen Übersetzungsmaschine“, was den Prozess verlangsamt und fehleranfällig macht.

4.2.4 Limitierungen bestehender Standard-Tools

Aktuelle Standardlösungen wie das SAP Migration Cockpit decken primär die rein technische Datenübernahme ab, lassen bei der Logik-Transformation jedoch Lücken. Der Experte äußert den Bedarf nach einer prozessualen Unterstützung auf „beratender Flugebene“, die Zusammenhänge aufzeigt und Optimierungspotenziale identifiziert. Aktuell fehlen Werkzeuge, die historischen Code automatisch kommentieren, komplexe Z-Functions analysieren oder proaktive Ratschläge zur Modellstrukturierung geben. Eine rein statische Code-Konvertierung erweist sich als unzureichend.

4.2.5 Manueller Aufwand und Fehleranfälligkeit im Testprozess

Die Qualitätssicherung stellt einen signifikanten Flaschenhals dar. Da bestehende Test-Tools logische Änderungen zwischen Alt- und Neusystem oft nicht korrekt interpretieren können, müssen migrierte Modelle weitgehend manuell getestet werden. Dieser händische Abgleich ist extrem aufwendig und birgt in der Validierungsphase ein hohes Risiko für Flüchtigkeitsfehler.

4.2.6 Zeitliche Metriken und Identifikation von Flaschenhälsen

Die Schätzung des zeitlichen Migrationsaufwands variiert je nach Modellkomplexität erheblich. Während ein durchschnittliches Modell laut dem Experten in drei bis vier Werktagen überführt werden kann, erfordert allein die Erstellung von Constraint-Tabellen aus bestehenden Vorbedingungen bei hochkomplexen Modellen einen mehrmonatigen Arbeitsaufwand. Selbst nach dieser zeitintensiven manuellen Übertragung weisen die generierten Tabellen häufig noch hohe Fehlerraten auf. Dies verdeutlicht, dass die manuelle Migration mit zunehmender Komplexität der Legacy-Modelle zu einem extrem langwierigen und fehleranfälligen Prozess wird.

Zur Definition der Baseline differenziert der Experte zudem zwischen zwei Vorgehensweisen. Beim Greenfield-Ansatz (kompletter Neuaufbau) verteilen sich die Aufwände auf die Merkmalswelt (20%), die Analyse (40%) und die Umsetzung (40%). Da sich ein derartiger manueller Neuaufbau informationstechnisch kaum automatisieren lässt, fokussiert sich der Prototyp auf die direkte Migration. Bei diesem Ansatz stellt die Umwandlung der Vorbedingungen in Constraints den primären prozessualen Flaschenhals dar: Dieser Arbeitsschritt bindet schätzungsweise 66% der Gesamtarbeitszeit und bildet somit den zentralen Ansatzpunkt für die technologische Unterstützung.

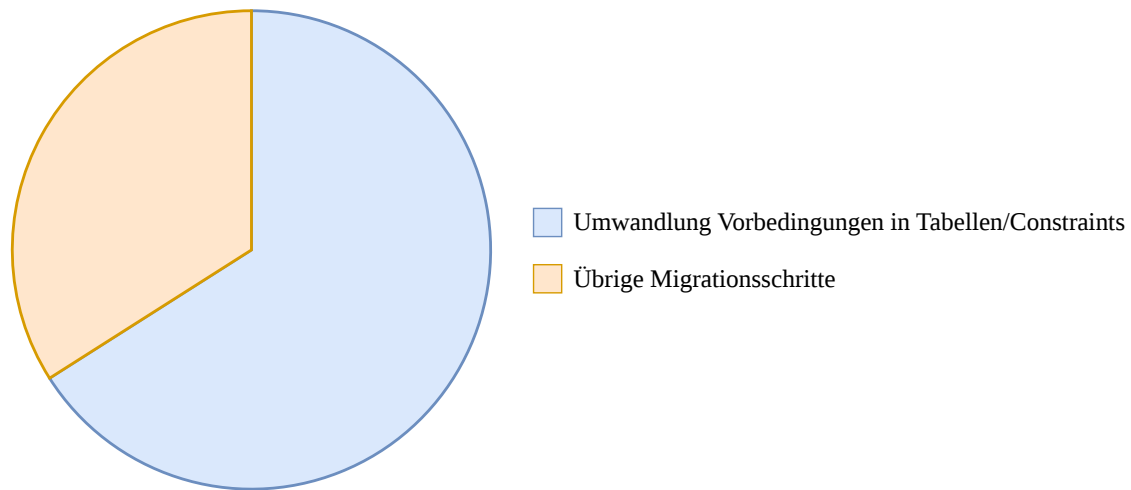


Abbildung 4.1: Zeitaufwand bei der direkten Migration von LO-VC zu AVC (Schätzwert nach Experteninterview).

4.3 Implikationen für die Systemarchitektur (Ideales Zielbild)

Aus den empirischen Befunden lassen sich konkrete Anforderungen an die Systemarchitektur ableiten. Dieser Abschnitt stellt ausschließlich die Implikationen dar, die direkt aus dem Interview resultieren. Sie repräsentieren den maximalen *Business Need* der Praxis und skizzieren das theoretische Idealbild des Systems.

4.3.1 Funktionale Anforderungen aus der Expertenperspektive

Aus den praktischen Hürden ergeben sich folgende zentrale funktionale Wünsche an das Systemdesign:

- **Semantische Analyse und Code-Bereinigung:** Das System sollte ungenutzte Merkmale und Prozeduren automatisch identifizieren, um Legacy-Code im Vorfeld zu bereinigen und die kognitive Last zu senken.
- **Automatisierte Logik-Transformation:** Um den zeitlichen Flaschenhals zu beheben, muss das System Tabellen und Constraints automatisiert generieren und die logische Restrukturierung von Vorbedingungen eigenständig übernehmen.
- **Konsistenzprüfung im Testprozess:** Um die Validierung zu erleichtern, wird eine Funktion benötigt, die alte Testkonfigurationen auf die Zielmodelle mappt, logische Abweichungen erkennt und den manuellen Testaufwand reduziert.

4.3.2 Architektonische Paradigmen und Beratungsansatz

Der Paradigmenwechsel in der Modellierungstechnik erfordert zudem spezifische Architektureigenschaften:

- **Interaktive Assistenz (Consulting-Ansatz):** Das System darf nicht nur als deterministischer Konverter agieren. Es soll Design-Entscheidungen begründen und den Nutzer beratend unterstützen, um den Transformationskontext transparent zu machen.
- **Einhaltung von AVC-Modellierungsrichtlinien:** Die Architektur muss standardkonformen AVC-Code erzeugen und Performance-Best-Practices (z.,B. nativen SAP-Standard statt veralteter Z-Funktionen) direkt berücksichtigen, um eine fehlerfreie Lauffähigkeit zu garantieren.

Die Tabelle 4.1 fasst die empirischen Befunde und die daraus abgeleiteten Systemanforderungen abschließend zusammen.

4.3.3 Quantitative Zielvorgaben für die Systemvalidierung

Die erhobenen Zeitmetriken definieren die Erfolgskriterien für die spätere Evaluierung. Das System muss sich also bei mittleren Modellen gegen die manuelle Baseline von drei bis vier Tagen pro Modell beweisen. Insbesondere soll gezeigt werden, dass die automatisierte Unterstützung den Hauptflaschenhals der Vorbedingungs-Transformation (66% des Aufwands) messbar reduziert und dem Entwickler eine schnellere Migration ermöglicht.

Empirische Befunde (Ist-Zustand, Kap. 4.2)

Systemimplikationen (Ideales Zielbild, Kap. 4.3)

4.2.1 Kognitive Altlasten:

Toter Code und ungenutzte Merkmale erschweren die 1:1 Konvertierung und belasten den Entwickler.

Semantische Analyse & Code-Bereinigung:

Automatische Identifikation ungenutzter Elemente zur Senkung der kognitiven Last im Vorfeld.

4.2.6 & 4.2.3 Flaschenhals Vorbedingungen:

Manuelle Auswertung und Restrukturierung binden 66 % der Gesamtarbeitszeit bei der direkten Migration.

Automatisierte Logik-Transformation:

Eigenständige Generierung von Tabellen und Constraints zur Behebung des zeitlichen Flaschenhalses.

4.2.5 Fehleranfälligkeit im Testprozess:

Händischer Abgleich migrierter Modelle ist extrem aufwendig und birgt hohes Risiko für Flüchtigkeitsfehler.

Konsistenzprüfung im Testprozess:

Mapping alter Testkonfigurationen auf Zielmodelle zur Erkennung logischer Abweichungen.

4.2.4 Limitierungen von Standard-Tools:

Reine Code-Konvertierung ohne proaktive Ratschläge auf einer „beratenden Flugebene“ reicht nicht aus.

Interaktive Assistenz (Consulting-Ansatz):

Das System muss über deterministische Konvertierung hinausgehen und Design-Entscheidungen begründen.

4.2.2 Paradigmenwechsel & Clean Core:

Zwang zu klassifizierten Merkmalen und Verbot proprietärer Eigenentwicklungen (z. B. Z-Functions).

Einhaltung von AVC-Modellierungsrichtlinien:

Generierung von standardkonformem Code unter direkter Berücksichtigung von Performance-Best-Practices.

Tabelle 4.1: Ableitung der funktionalen und architektonischen Systemanforderungen (Kap. 4.3) aus den empirisch identifizierten Praxis-Hürden (Kap. 4.2).

Kapitel 5

Machbarkeitsanalyse und Selbstversuch

Nachdem das Experteninterview im vorangegangenen Kapitel die subjektiven Praxisprobleme und die zeitliche Baseline definiert hat, verlagert dieses Kapitel den Fokus auf die technische Ebene der rohen LO-VC-Datenstrukturen. Als zweiter empirischer Baustein zur Analyse des *Environments* nach Hevner et al. (2004) verfolgt es zwei konkrete Ziele: Zunächst prüft eine Machbarkeitsanalyse (Scoping), welche der idealisierten Praxisanforderungen sich technologisch im Rahmen dieser Arbeit überhaupt automatisieren lassen. Darauf aufbauend dient ein praktischer Selbstversuch dazu, die manuelle Code-Migration Schritt für Schritt zu durchlaufen und exakte syntaktische Transformationsregeln zu extrahieren. Diese praktischen Erkenntnisse liefern die methodische Blaupause, um im Anschluss die Architektur und die System-Prompts des Multi-Agenten-Systems fundiert ableiten zu können.

5.1 Technologischer Realitätsabgleich und empirische Scope-Definition

Die im vorherigen Kapitel beschriebenen Praxisanforderungen (vgl. Kapitel 4.3) skizzieren das Idealbild eines zukünftigen Migrationssystems. Nach dem Paradigma des *Design Science Research* (Hevner et al., 2004) dürfen diese Wünsche jedoch nicht ungeprüft in die Systementwicklung übernommen werden. Es ist vorab eine Machbarkeitsprüfung (*Feasibility Check*) notwendig, um den technologischen und funktionalen Umfang (*Scope*) des Prototyps klar abzugrenzen.

Dieser Abschnitt evaluiert die fünf Kernimplikationen aus Kapitel 4.3. Es wird festgelegt, welche Anforderungen im folgenden praktischen Selbstversuch auf Code-Ebene untersucht und anschließend in die Multi-Agenten-Architektur integriert werden:

5.1.1 Evaluation der funktionalen Praxisanforderungen

1. **Konsistenzprüfung im Testprozess (Out-of-Scope):** Die automatisierte Überführung historischer Testkonfigurationen würde Migrationsprojekte erheblich entlasten. Die Umsetzung in diesem Proof-of-Concept ist jedoch *Out-of-Scope*. Eine solche Funktion erfordert eine tiefe Integration in kundenspezifische Testumgebungen und unstrukturiertes Domänenwissen über Altsysteme. Da sich diese Umgebung im Prototyp nicht simulieren lässt, können KI-Agenten die Validierung nicht verlässlich übernehmen. Ein dedizierter „Tester-Agent“ wird daher verworfen.
2. **Semantische Analyse und Code-Bereinigung (In-Scope):** Funktional obsolete Altlasten lassen sich direkt im Code identifizieren. Da diese Bereinigung den Entwickler stark entlastet, wird sie in den Scope aufgenommen. Der Selbstversuch dient dazu, wiederkehrende Muster von „totem Code“ zu erkennen. So können dem zukünftigen Analysten-Agenten entsprechende Erkennungsregeln mitgegeben werden.
3. **Automatisierte Logik-Transformation (In-Scope – Kernfokus):** Die Baseline zeigt, dass die manuelle Umstrukturierung von Vorbedingungen etwa zwei Drittel des Gesamtaufwands ausmacht. Diese Anforderung bildet den Kern der Systementwicklung. Der Selbstversuch konzentriert sich auf diesen Bereich, um die komplexen Pfade der Code-Umwandlung zu durchdringen und in formale Übersetzungsregeln zu überführen.

5.1.2 Evaluation der architektonischen Paradigmen

1. **Interaktive Assistenz / Consulting-Ansatz (Fokus auf Explainability):** Ein frei agierender „Consultant-Agent“, der in einem offenen Dialog strategische Designentscheidungen diskutiert, übersteigt den Rahmen dieses Prototyps. Um dem Praxisbedarf nach Transparenz dennoch gerecht zu werden, wird dieser Ansatz auf das Prinzip der *Explainability* fokussiert. Das System wird so konzipiert, dass es dem Entwickler seine getroffenen architektonischen und syntaktischen Entscheidungen schlüssig in Form von Kommentaren oder kurzen Begründungen darlegt. So bleibt für den Menschen stets nachvollziehbar, warum das System bestimmte Übersetzungspfade gewählt hat.
2. **Einhaltung von AVC-Modellierungsrichtlinien (In-Scope):** Das System muss standardkonformen AVC-Code erzeugen. Obsolete Elemente, wie klassische Z-Funktionen, müssen zwingend durch native AVC-Standardfunktionen ersetzt werden. Der Selbstversuch hilft dabei, die genauen semantischen Äquivalente im AVC-Standard-Repository zu ermitteln.

5.1.3 Synthese des finalen System-Scopes

Dieser Realitätsabgleich schärft den Fokus der weiteren Forschung. Das Sampling und die qualitative Analyse im folgenden Selbstversuch (vgl. Kapitel ?? ff.) konzentrieren sich folglich ausschließlich auf die **Code-Bereinigung**, die **Logik-Transformation** der Vorbedingungen sowie die Integration von **Explainability** zur Nachvollziehbarkeit der maschinellen Entscheidungen.

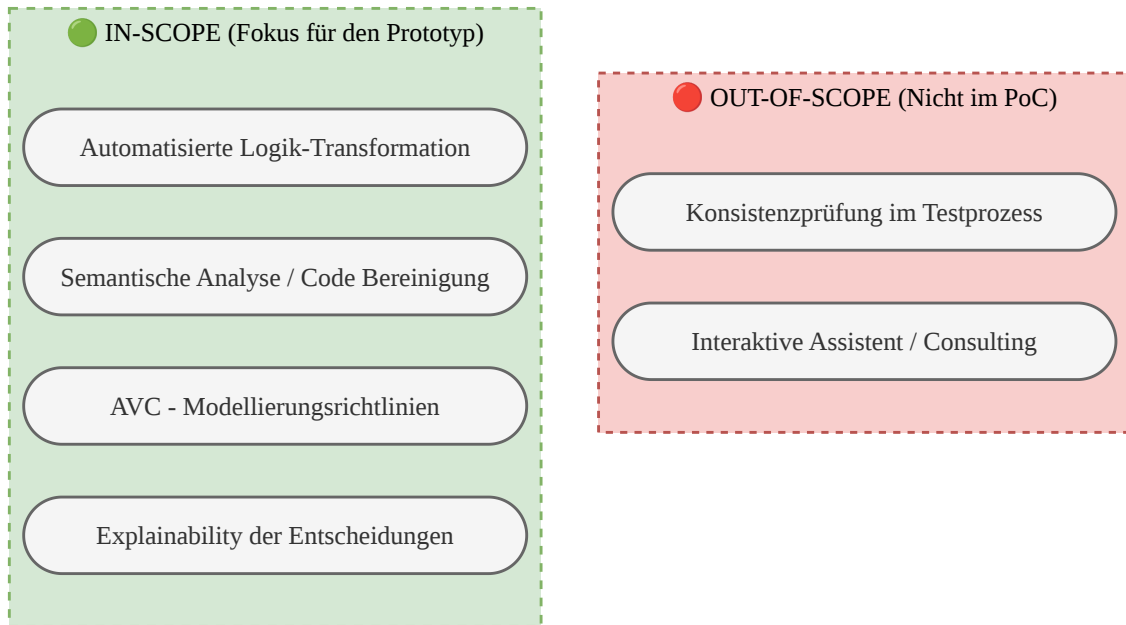


Abbildung 5.1: Scope-Matrix des Prototyps

Auch die geplante Multi-Agenten-Architektur wird entsprechend eingegrenzt: Das System verzichtet auf autonome Testumgebungen und interaktive Chatbot-Komponenten. Stattdessen fokussiert sich der Prototyp auf ein spezialisiertes, arbeitsteiliges Kernteam von KI-Agenten, welches die Phasen der Datenanalyse, der architektonischen Restrukturierung und der Code-Generierung abdeckt. Die exakte Definition und Ausgestaltung dieser spezifischen Agentenrollen (Version 1) wird direkt aus den Ergebnissen des nun folgenden praktischen Selbstversuchs abgeleitet.

5.2 Methodisches Setup und Durchführung

Um die abstrakten Anforderungen aus der Praxis in konkrete agentische Rollen und System-Prompts zu übersetzen, wurde der kognitive Transformationsprozess durch einen praktischen Selbstversuch auf Code-Ebene analysiert. Anstatt dieses introspektive Vorgehen als klassische Fallstudie zu deklarieren, nutzt diese Arbeit das etablierte Rahmenwerk nach Runeson und Höst (2009) als methodische Schablone zur strukturierten Protokollierung. Die klassischen Prinzipien der Datenerhebung und Auswertung wurden dabei zielgerichtet

adaptiert, um die eigenen kognitiven Schritte beim Programmieren nachvollziehbar zu dokumentieren. Das methodische Vorgehen definiert sich durch folgende Kernkriterien:

Zielsetzung und Forschungsfragen (Objective & Research Questions): In der Softwaretechnik reicht es für die Automatisierung von Prozessen oft nicht aus, nur den ursprünglichen und den finalen Code zu betrachten. Der eigentliche Wert liegt in den kognitiven Zwischenschritten des Entwicklers – dem impliziten Erfahrungswissen (*Tacit Knowledge*), das bei der Problemlösung meist unstrukturiert im Hintergrund abläuft (Seaman, 1999). Das Ziel dieses Selbstversuchs ist es, dieses unbewusste Wissen greifbar zu machen und den kognitiven Übersetzungsprozess (Baseline) methodisch zu dekonstruieren, um daraus die Architektur und die System-Prompts des Multi-Agenten-Systems ableiten zu können. Somit beantwortet dieser Selbstversuch sowohl einen Teil der ersten Forschungsfrage als auch die zweite Forschungsfrage.

Fallauswahl und Selektionsstrategie (The Case & Selection Strategy): Die Datengrundlage für diesen Selbstversuch bildete ein reales Kundenprojekt der *msg systems ag*. Aufgrund strenger Datenschutzvorgaben, Geschäftsgeheimnissen und stark reglementierter Zugriffsrechte auf historische Altdaten war die Auswahl pragmatisch auf dieses spezifische, verfügbare Projekt beschränkt. Ähnlich wie im Kapitel 4 wird also auch hier im Einklang mit Runeson und Höst (2009) primär nach Verfügbarkeit ausgewählt. Aus diesem Projekt wurden fünf Merkmale mitsamt ihrem historisch gewachsenen Beziehungswissen für die manuelle Migration ausgewählt. Die Begrenzung auf genau fünf Merkmale ergab sich aus dem praktischen Verlauf: Nach der Bearbeitung zeigte sich, dass sich die grundlegenden kognitiven Schritte stetig wiederholten und inhaltliche Sättigung erreicht war.

Datenerhebungsmethode (Methods): Das methodische Vorgehen stützt sich auf die Richtlinien für explorative Fallstudien nach Runeson und Höst (2009) und die Prinzipien der qualitativen Analyse nach Seaman (1999). Um das von *Tacit Knowledge* zu erheben, schlüpfte der Autor in die Rolle des Entwicklers und migrierte die realen LO-VC-Daten manuell. Anstatt dabei eine hochkomplexe theoretische Analysemethode anzuwenden, wurde der Migrationsprozess systematisch so protokolliert, wie er in der Praxis kognitiv abläuft. Jeder Schritt wurde in drei klar getrennten Phasen dokumentiert:

1. **Bestandsaufnahme (Input):** Festhalten der historischen LO-VC-Strukturen und Vorbedingungen.
2. **Kognitive Analyse (Gedankengänge):** Explizite schriftliche Dokumentation der Überlegungen (z.,B. Identifikation von veraltetem Code oder Architekturentscheidungen zur Bündelung).
3. **Code-Synthese (Output):** Erstellung des aus den Gedankengängen resultierenden AVC-Codes.

Datenauswertung (Data Analysis): Die Auswertung erfolgte durch den direkten analytischen Vergleich von Input, dem eigenen Denkprozess und dem finalen Output.

Durch dieses Verfahren ließen sich wiederkehrende Handlungsmuster beim Programmieren identifizieren. Diese Dokumentation der eigenen Denkschritte bildet die direkte Auswertungsvorlage, um im nächsten Schritt die konkreten agentischen Rollen und Instruktionen (Prompts) für das *CrewAI*-System abzuleiten.

5.3 Ergebnisse des Selbstversuchs und Ableitung der kognitiven Architektur

Die vollständigen Protokolle der manuellen Migration – inklusive Input-Code, kognitiven Zwischenschritten und finalem AVC-Code – sind aus Gründen der Lesbarkeit im Anhang ?? dokumentiert. Dieser Abschnitt fasst die zentralen Erkenntnisse der Fallstudie zusammen und leitet daraus die Handlungsregeln für die Multi-Agenten-Architektur ab.

5.3.1 Kognitive Phasen der manuellen Migration

Die Analyse der Protokolle zeigt, dass die Transformation von LO-VC nach AVC weit mehr ist als ein einfacher Syntax-Austausch.

Der Entwickler durchläuft während der Migration unbewusst ein dreistufiges kognitives Modell. Eine bloße 1:1-Übersetzung würde in der neuen AVC-Engine zu fehlerhaftem oder ineffizientem Code führen. Folgende drei Phasen bilden die Blaupause für das zu entwickelnde System:

1. **Strukturelle und semantische Bereinigung (Die Analysten-Perspektive)** Der historische Kontext muss vor der Transformation bereinigt werden. Die untersuchten Modelle enthielten häufig verwaiste Abhängigkeiten, funktionslosen Legacy-Code, alte Entwicklerkommentare oder logische Redundanzen. Eine rigorose Filterung dieser obsoleten Elemente ist der erste notwendige Schritt, um Fehler im weiteren Verlauf zu vermeiden.
2. **Architektonische Design-Entscheidungen (Die Architekten-Perspektive)** Nach der Bereinigung muss die strukturelle Zielform festgelegt werden. Dies erwies sich in der Fallstudie als der komplexeste Schritt:
 - Teilen sich verschiedene Einzelwerte dieselben Vorbedingungen, ist es ineffizient, diese einzeln zu übersetzen. Solche Muster müssen erkannt und zu performanten Variantentabellen aggregiert werden.
 - Sind Tabellen aufgrund komplexer Ungleichungen nicht optimal, müssen die Werte anders gebündelt werden (z. B. durch den IN-Operator).
 - Zudem muss erkannt werden, welche alten Strukturen (wie der INFERENCES-Block) im AVC gänzlich entfallen.

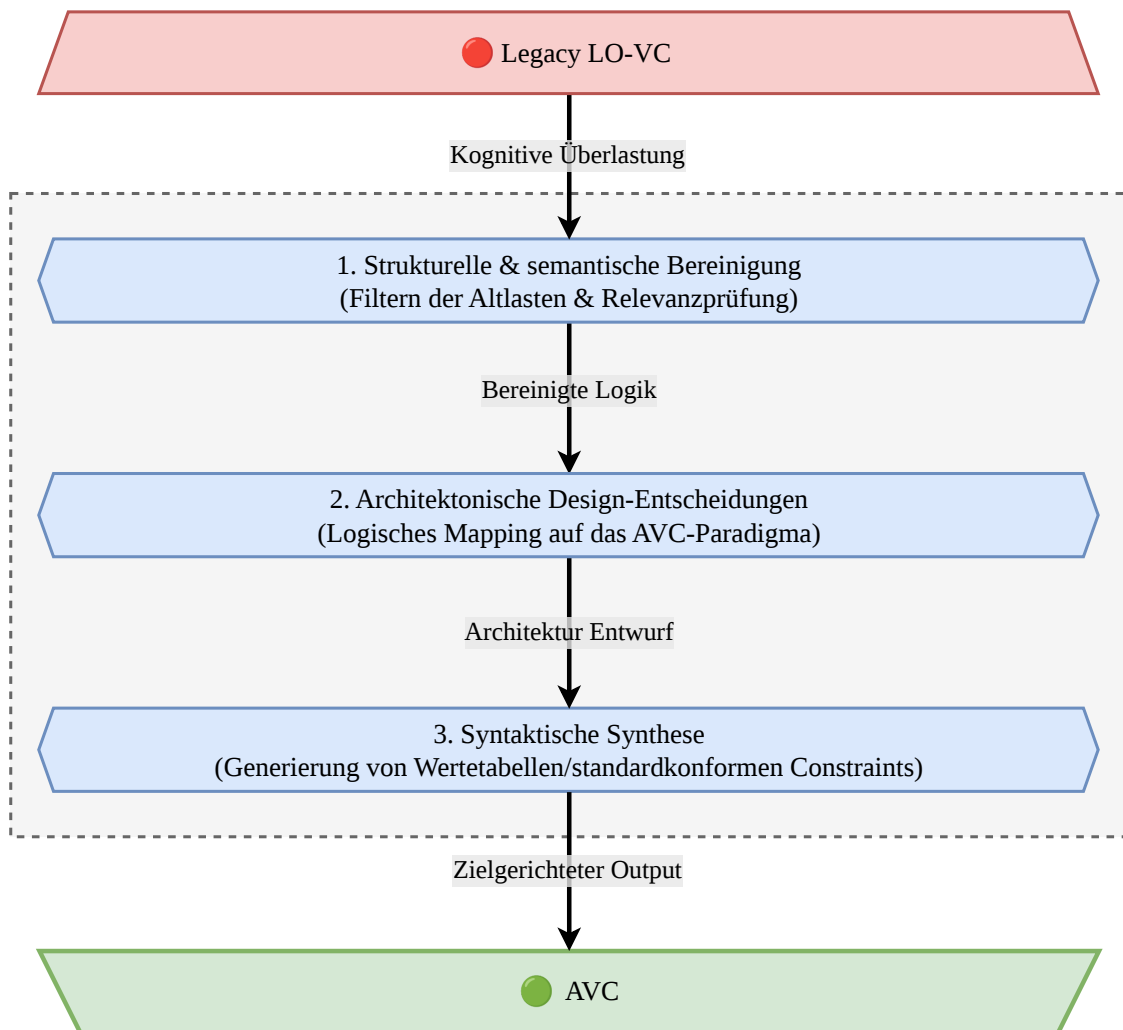


Abbildung 5.2: Das dreistufige kognitive Modell der manuellen Code-Migration.

3. Syntaktische Synthese (Die Entwickler-Perspektive)

Erst wenn dieser logische Bauplan steht, folgt die eigentliche Programmierung nach den strikten AVC-Regeln. Das umfasst das Schreiben der Restriktionen, die korrekte Pfadreferenzierung (z. B. `$self.` oder `$root.`) und das Ersetzen veralteter Operatoren (z. B. `ne` durch `<>`).

5.3.2 Ableitung der V1-Architektur und System-Prompts

Um diesen komplexen kognitiven Prozess maschinell abzubilden, ist ein einzelnes Sprachmodell (Single-Prompting) ungeeignet. Ein solches Modell wäre mit Bereinigung, Architekturentscheidung und exakter Formatierung gleichzeitig überfordert. Die Ergebnisse des Selbstversuchs erfordern stattdessen eine arbeitsteilige Multi-Agenten-Architektur.

Das in Kapitel 6 konzipierte System wird exakt nach diesem Vorbild aufgebaut: Ein kollaboratives Team aus drei KI-Agenten, unterstützt von einem deterministischen Validierungstool. Aus der Fallstudie lassen sich folgende initiale Rollen und Instruktionen (Prompts) ableiten:

Rolle 1: LO-VC Data Analyst & Cleaner

Ziel: Bereinigung der

rohen JSON-Eingabedaten. **Handlungsregel:** Analysiere komplexe Vorbedingungen. Identifiziere und ignoriere toten Code und alte Kommentare. Erstelle aus den gültigen Werten eine saubere Liste der steuernden Merkmale.

Rolle 2: AVC Table Architect

Ziel: Entwurf eines logischen Bauplans.

Handlungsregel: Entscheide anhand der bereinigten Daten, welche Merkmale Tabellenspalten bilden. Erkenne identische Vorbedingungen und weise den nachfolgenden Agenten an, diese effizient zusammenzufassen (z. B. über `IN()`-Operatoren oder Tabellen).

Rolle 3: AVC Syntax Coder (Synthesizer)

Ziel: Übersetzung des Bauplans in fehlerfreien AVC-Code.

Handlungsregel: Schreibe den finalen Code (TABLE-Aufrufe und RESTRICT-Constraints). Nutze die korrekten Objektreferenzen (wie `$self.`) und wende die aktuellen Standard-Operatoren an.

Diese empirisch hergeleitete Agentenstruktur bildet das Fundament für die anstehende technische Entwicklung des Systems.

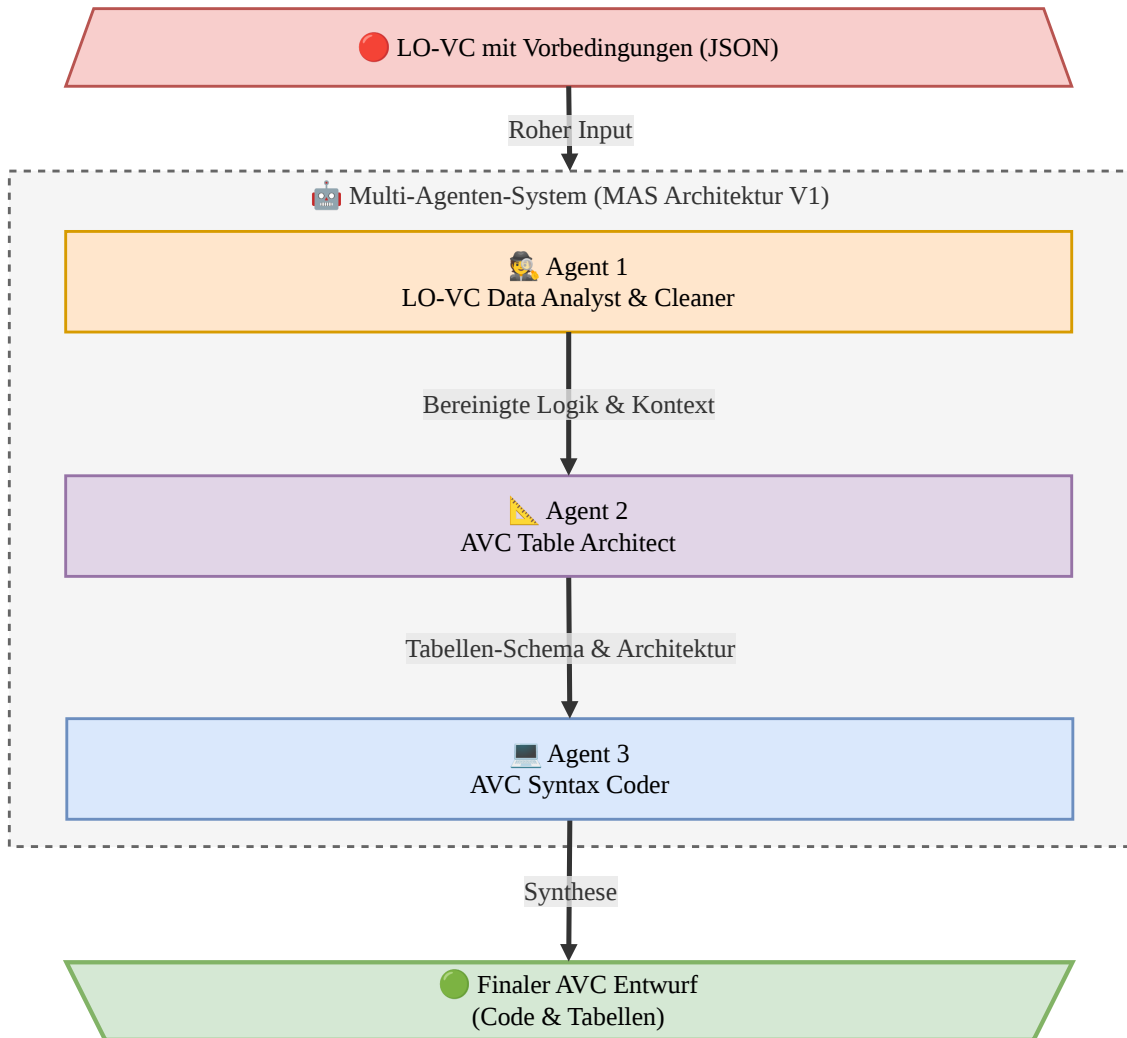


Abbildung 5.3: Architektur-Skizze des Multi-Agenten-Systems (V1).

Kapitel 6

Systemdesign und Prototypenentwicklung

Nachdem in den beiden vorangegangenen Abschnitten die Zielarchitektur des Multi-Agenten-Systems konzeptionell aus der Anforderungsanalyse hergeleitet wurde, dokumentiert dieses Kapitel die informationstechnische Umsetzung des Prototyps. Die Entwicklung gliedert sich hierbei in zwei Phasen: Zunächst wird die Technologieauswahl und Infrastruktur begründet. Darauf aufbauend wird im Sinne des *Design Science Research* (DSR) die iterative, evolutionäre Entwicklung der Systemarchitektur dokumentiert, welche durch kontinuierliche Evaluierungszyklen zur finalen Pipeline führte.

6.1 Technologieauswahl und Infrastruktur

Die Auswahl der genutzten Frameworks und Plattformen richtet sich nach den internen Vorgaben der *msg systems ag* an Datenschutz und IT-Sicherheit sowie nach grundlegenden Prinzipien der Softwareentwicklung wie Wartbarkeit und Flexibilität. Das technische Fundament des Prototyps besteht aus drei Komponenten: dem Orchestrierungs-Framework *CrewAI*, dem *SAP AI Core* als sicherer Ausführungsumgebung und *liteLLM* als standardisierter Schnittstelle zur Modellanbindung.

6.1.1 Orchestrierungs-Framework: CrewAI

Laut Liu et al. (2025) gehören LangGraph, AutoGen und CrewAI zu den aktuell etabliertesten Frameworks für Multi-Agenten-Systeme. Für den Prototyp in dieser Arbeit fiel die Wahl auf *CrewAI*. Diese Entscheidung ergab sich primär aus den strukturellen Anforderungen der Code-Migration sowie dem Ziel, die Entwicklungszeit effizient zu halten. Die in Kapitel 2 vorgestellten Konzepte wie dynamische *ReAct*-Schleifen oder offene Agenten-Dialoge (wie sie beispielsweise *AutoGen* nutzt) eignen sich vor allem für ergebnisoffene Problemlösungen. Für die Migration von SAP-Constraint-Logik sind sie jedoch weniger zielführend. Da in

diesem Prototyp aufgrund der proprietären SAP-Umgebung kein direkter Compiler-Zugriff existiert, erhalten die Agenten kein verlässliches technisches Feedback. Freie, ungelenkte Interaktionsschleifen zwischen Agenten führen unter diesen Bedingungen schnell zu Halluzinationen oder Endlosschleifen. Die Aufgabenstellung erfordert stattdessen einen stark geführten, sequenziellen Ablauf. *CrewAI* setzt das in Kapitel 2.3.2 beschriebene „Fließband-Paradigma“ um (Singh, 2025). Die Agenten arbeiten hier in einer fest definierten Pipeline: Analyse, Architekturentwurf und Code-Synthese folgen strikt nacheinander. Diese Einschränkung der Freiheitsgrade ist bei der Code-Migration von Vorteil, da sie einen deterministischeren Prozess erzwingt und Fehler durch ungeplante Agenten-Interaktionen reduziert. Ein ähnlicher Workflow ließe sich zwar auch mit *LangGraph* abbilden, *CrewAI* bietet hierfür jedoch einen deutlich geringeren Programmieraufwand bei der grundlegenden Zustandsverwaltung. Auch wenn die Architektur auf dynamisches ReAct-Feedback verzichtet, kommen andere zentrale KI-Konzepte aus Kapitel 2 zum Einsatz. Da ein aufwändiges *Fine-Tuning* eigener Sprachmodelle für die spezifische SAP-Domäne den Rahmen der Arbeit sprengen würde, nutzt das System primär *In-Context Learning (ICL)* und *Few-Shot Prompting*. Diese Methoden sind direkt in die System-Prompts der Agenten integriert: Die Rollenbeschreibungen in *CrewAI* enthalten nicht nur Handlungsanweisungen, sondern auch konkrete Lösungsbeispiele (*Few-Shot*) für die korrekte AVC-Syntax, an denen sich die Sprachmodelle bei der Code-Generierung orientieren können.

6.1.2 Infrastruktur und Modellintegration: SAP AI Core und liteLLM

Bei der automatisierten Verarbeitung von industriellen Produktdaten und Konfigurationsmodellen (LO-VC) müssen interne Datenschutzvorgaben zwingend eingehalten werden. Da diese Modelle vertrauliches Firmenwissen enthalten, ist die direkte Übertragung an öffentliche APIs (wie die Standard-Schnittstelle von OpenAI) aus Sicherheitsgründen ausgeschlossen.

Um diese Vorgaben zu erfüllen, erfolgt die Anbindung der Sprachmodelle in diesem Prototyp über den *SAP AI Core*. Dies stellt sicher, dass die Datenverarbeitung in einer kontrollierten Systemumgebung stattfindet. Neben dem Datenschutz hat diese Entscheidung auch organisatorische Gründe: Da die *msg systems ag* den *SAP AI Core* bereits in anderen Projekten einsetzt, ist die nötige Infrastruktur bereits vorhanden. Dies erleichtert die technische Bereitstellung und ermöglicht es, die anfallenden Nutzungskosten standardisiert über bestehende Unternehmensverträge abzurechnen.

Ein weiterer technischer Aspekt betrifft die Anbindung der Modelle an das Agentensystem. Um flexibel auf technologische Änderungen reagieren zu können, soll der Prototyp nicht fest an einen einzelnen Anbieter oder ein spezifisches Modell gebunden werden (*Vendor Lock-in*).

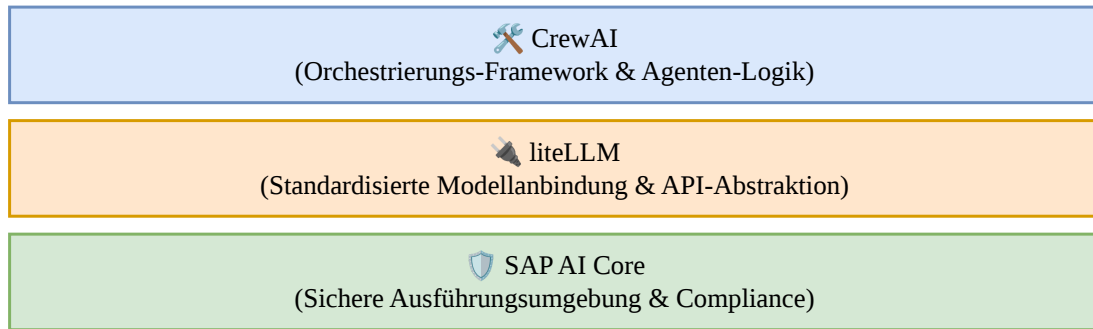


Abbildung 6.1: Schichtenarchitektur des technischen Fundaments.

Um dies zu verhindern, wird die Python-Bibliothek *liteLLM* genutzt und in das *CrewAI*-Framework integriert. Sie fungiert als standardisierende Schnittstelle zwischen den KI-Agenten und den Sprachmodellen. Dadurch wird der Quellcode der Multi-Agenten-Architektur vom verwendeten Modell entkoppelt. Das ermöglicht es, das zugrundeliegende Modell bei Bedarf mit minimalem Konfigurationsaufwand über den *SAP AI Core* auszutauschen, ohne den Programmcode der eigentlichen Agenten anpassen zu müssen.

6.2 Iterative Entwicklung des Prototyps (Design Science Research)

Gemäß dem *Design Science Research*-Ansatz nach Hevner et al. wurde das Multi-Agenten-System in iterativen Entwicklungs- und Testzyklen (*Build-and-Evaluate*) umgesetzt. Um die Ergebnisse objektiv vergleichen zu können, wurde in der gesamten Prototyping-Phase eine konstante Datenbasis verwendet. Diese bestand aus fünf Beispielsmerkmalen inklusive ihrer Ausprägungen und Vorbedingungen, die aus einem realen Kundenprojekt extrahiert wurden.

Die folgenden Abschnitte dokumentieren die schrittweise Weiterentwicklung der Systemarchitektur. In jeder Phase wurden bestehende Fehler und technische Limitierungen analysiert und behoben, bis die finale Architektur feststand.

6.2.1 Zyklus 1: Limitierungen einer rein agentenbasierten Architektur

Die erste Version des Prototyps (V1-Architektur) basierte auf der Annahme, dass ein Netzwerk aus drei KI-Agenten (Analyst, Architekt, Synthesizer) die JSON-Daten schrittweise verarbeiten kann. Der Prozess lief sequenziell pro Merkmal ab: Das System erhielt ein standardisiertes JSON-Dokument mit allen Merkmalen, Werten und Vorbedingungen. Die Agenten sollten die Daten nacheinander bearbeiten und am Ende ein strukturiertes

Zielformat ausgeben. Dieses Ergebnis sollte entweder aus einfachem AVC-Code bestehen (falls das System entscheidet, keine Tabelle zu generieren) oder aus einem Tabellenvorschlag inklusive des zugehörigen AVC-Aufrufs.

Die ersten Tests mit dieser grundlegenden Architektur zeigten jedoch deutliche Schwächen:

- **Mangelnde Reproduzierbarkeit:** Das System lieferte keine deterministischen Ergebnisse. Bei drei Durchläufen mit exakt derselben Datenbasis generierten die Agenten drei unterschiedliche Ausgaben.
- **Fehlendes Wissen:** Einem *Large Language Model* (LLM) lediglich per System-Prompt die Rolle eines SAP-AVC-Entwicklers zuzuweisen, reichte nicht aus. Das Modell kannte die spezifische Syntax nicht ausreichend, verlor schnell den fachlichen Kontext und nutzte falsche Bezeichnungen für Merkmale und Werte.
- **Probleme bei der Tabellenbildung:** Die Hauptaufgabe – zu bewerten, wann eine Variantentabelle sinnvoll ist und wann nicht – überforderte das System. Besonders in Fällen, in denen innerhalb eines Merkmals nur bestimmte Werte für eine Tabelle geeignet waren und andere isoliert behandelt werden mussten, konnte das LLM keine korrekten und konsistenten Entscheidungen treffen.

Aus diesen initialen Testergebnissen ließ sich ableiten, dass die komplexe Strukturierungs- und Übersetzungsaufgabe nicht allein durch ein generisches Agenten-Setup gelöst werden kann. Um die mangelnde Zuverlässigkeit und das fehlende domänenspezifische Fachwissen der Modelle zu kompensieren, waren für die folgende Iteration gezielte funktionale und inhaltliche Erweiterungen des Prototyps erforderlich.

6.2.2 Zyklus 2: Code-Refaktorisierung, Wissensbasis und strukturelle Halluzinationen

Um den identifizierten Schwächen aus dem ersten Zyklus zu begegnen, wurden in der zweiten Iteration mehrere methodische und strukturelle Anpassungen am Prototyp vorgenommen. Zunächst wurde der Quellcode in verschiedene Module (`main.py`, `agents.py`, `utils.py`) aufgeteilt, um die Übersichtlichkeit und Wartbarkeit der Anwendung zu verbessern. Zudem wurden die Aufgabenbeschreibungen (Prompts) der einzelnen Agenten inhaltlich deutlich geschärft.

Um das im ersten Durchlauf fehlende Syntaxwissen auszugleichen, wurde ein separates Wissensdokument angelegt, das dem Synthesizer-Agenten als Referenz für gültigen AVC-Code dienen sollte. Zur besseren Überprüfbarkeit der generierten Ergebnisse wurde außerdem eine Hilfsfunktion in die `utils.py` integriert. Diese formatiert den Text-Output des

Large Language Models so um, dass nach jedem Durchlauf automatisch ein strukturiertes Excel-Dashboard erstellt wird.

Die Evaluierung dieses angepassten Systems zeigte zunächst einen organisatorischen Erfolg: Die Aufbereitung der Ergebnisse im Excel-Dashboard funktionierte zuverlässig und erleichterte die manuelle Code-Prüfung erheblich. Bei der inhaltlichen Logikgenerierung traten jedoch neue Probleme auf:

- **Anhaltende Stochastik:** Trotz der geschärften Prompts blieb das System unzuverlässig. Drei identische Durchläufe produzierten weiterhin drei unterschiedliche Ergebnisse.
- **Übergenerierung und Halluzinationen:** Das System verstand nun zwar grundsätzlich den Auftrag, zwischen Variantentabellen und isolierten *IF*-Bedingungen (Standalone) zu unterscheiden. Allerdings zeigte das LLM die Tendenz, zwanghaft für jedes Merkmal riesige Tabellen generieren zu wollen, in die alle Werte gequetscht wurden. Dabei verlor das Modell den Bezug zum JSON-Input: Es erfand Spaltennamen (teilweise aus einfachen Syntax-Fragmenten), halluzinierte nicht-existente Tabelleneinträge und behalf sich mit Platzhaltern, um die fehlerhaften Tabellenstrukturen künstlich aufrechtzuerhalten.
- **Problem des festverdrahteten Domänenwissens:** Ein konzeptioneller Fehler zeigte sich beim Umgang mit spezifischen Kundendaten. Das Herausfiltern des Wertes „specified dummy“ (ein kundenindividueller Workaround für obsoleten Code) war in dieser Phase noch fest im System-Prompt des Agenten einprogrammiert (Hardcoding).

Um die mangelnde Flexibilität beim Domänenwissen sofort zu adressieren, wurde noch innerhalb dieses Zyklus ein externes Konfigurationsdokument eingeführt. Der Entwickler kann hier projektbezogene Sonderregeln dynamisch hinterlegen, wodurch das Kernsystem domänenunabhängig wird. Begleitend wurden die System-Prompts der Agenten sowie das Dokument für die AVC-Syntax weiter detailliert, in der Hoffnung, die Präzision der Tabellengenerierung zu erhöhen.

Die erneute Prüfung dieser spezifischen Anpassungen brachte jedoch keine signifikante Verbesserung. Die Ergebnisse blieben bei mehrfacher Ausführung weiterhin inkonsistent, waren fachlich fehlerhaft und von den beschriebenen strukturellen Halluzinationen geprägt.

Die abschließende Erkenntnis aus Zyklus 2 zeigte unmissverständlich: Ein Sprachmodell ist selbst mit ausgelagertem Domänenwissen, externen Hilfsdokumenten und maximal geschärften Rollen nicht in der Lage, deterministische Datenstrukturen (wie Tabellen) konsistent und fehlerfrei aus einem JSON zu extrahieren.

6.2.3 Zyklus 3: Regelbasierte Vorverarbeitung und Domänenunabhängigkeit

Die Tests zeigten, dass die architektonische Entscheidungsfindung (Tabelle vs. Einzelbedingung) für das Sprachmodell zu komplex war. Um die Aufgabe zu vereinfachen, musste geklärt werden, ob es für die Erstellung von Variantentabellen klare, deterministische Regeln gibt. Die Rücksprache mit einem Fachexperten bestätigte dies: Eine Tabelle ist in der industriellen Praxis nur dann sinnvoll, wenn direkte Gleichsetzungsbedingungen vorliegen (z. B. `Merkmal_A = '1'`). Bei Ungleichungen (z. B. `<>`) verbietet sich eine Tabelle. Zudem lohnt sich der Aufbau einer Tabelle meist erst, wenn mehr als zwei Merkmale involviert sind, um eine Fragmentierung in viele Kleinsttabellen zu vermeiden.

Ein zweiter wichtiger Punkt aus der Absprache mit dem Experten betraf den Umgang mit der `specified`-Syntax. Diese prüft, ob ein Merkmal im Material überhaupt bewertet wurde. Der Kunde nutzte dies als Workaround, um obsoleten Code (Dummy-Werte) zu blockieren, da diese Dummy-Merkmale im Material nie klassifiziert waren. Um dieses harte Domänenwissen nicht mehr im Agenten-Prompt festschreiben zu müssen, wurde ein vorgeschaltetes Python-Skript entwickelt. Dieses Skript liest zunächst das gesamte JSON-Dokument ein und erstellt eine vollständige Liste aller im Material existierenden Merkmale. Mit dieser Liste kann das System im Vorfeld deterministisch prüfen, ob ein Ausdruck wie `specified(Merkmal_X)` wahr oder falsch ist, und den entsprechenden Code-Block automatisch verarbeiten oder ignorieren.

Zur besseren Fehleranalyse wurden ab dieser Phase zudem dedizierte Debugging-Dateien generiert, um die Zwischenergebnisse der Vorverarbeitung gezielt prüfen zu können, ohne immer den finalen Code abwarten zu müssen. Eines dieser Zwischenergebnisse war das gezielte Markieren von unklassifizierten Merkmalen durch den Python-Filter, um dem Agenten die syntaktische Verarbeitung zu erleichtern.

Die Evaluierung dieses Zyklus zeigte spürbare Verbesserungen: Die Ergebnisse wirkten weniger zufällig, und die Trennung zwischen Tabellenvorschlägen und *IF*-Bedingungen funktionierte teilweise. Dennoch traten bei mehrfachen Durchläufen weiterhin Halluzinationen und Inkonsistenzen auf. Zudem zeigte sich eine neue strukturelle Herausforderung: Das System erzeugte redundante Tabellen. Teilweise waren neu generierte Tabellen exakte Duplikate, teilweise handelte es sich um logische Teiltabellen (Sub-Tabellen), die inhaltlich bereits durch größere Tabellen abgedeckt waren.

6.2.4 Zyklus 4: Die finale neuro-symbolische Architektur und funktionale Abgrenzung

Aufgrund der begrenzten Bearbeitungszeit der Masterarbeit mussten in Bezug auf diese Tabellen-Redundanzen und die Syntax-Prüfung klare Grenzen für den Funktionsumfang (Scope) des Prototyps gezogen werden:

- **Tabellen-Redundanzen:** Das Erkennen und Eliminieren von logischen Teiltabellen erfordert komplexe mathematische Berechnungen (z. B. durch einen SAT-Solver). Da dies zeitlich nicht umsetzbar war, verbleibt dieses Problem in der *Future Work*. Umgesetzt wurde lediglich ein Modul, das exakt identische Tabellen zusammenführt (Join). Obwohl auch dies deterministisch über Python lösbar wäre, wurde es aus Zeitgründen als Agenten-Aufgabe im System belassen.
- **Syntax-Validierung:** Eine fehlerfreie syntaktische Prüfung von AVC-Code erfordert idealerweise einen *Abstract Syntax Tree* (AST) Parser. Auch dies wurde in die *Future Work* verschoben. Die Validierung der häufigsten Fehler (z. B. fehlende `$self`-Präfixe) übernimmt im finalen Prototyp stattdessen ein dedizierter KI-Agent mit rudimentären Prüfregelein.

Die Notwendigkeit dieser deterministischen Vorverarbeitung in Python bestätigt exakt die Intuition des Fachexperten aus dem initialen Interview (vgl. Anhang, Z. 42), der klassische Parser-Ansätze für strikte Regelwerke empfahl. Der iterative DSR-Prozess hat diese Praxis-Expertise empirisch validiert: Reine KI-Agenten scheitern an harten Constraint-Entscheidungen (Tabellenbau). Eine hybride (neuro-symbolische) Architektur ist zwingend erforderlich, wenngleich die Rollenverteilung zwischen Determinismus und KI in zukünftigen Iterationen weiter in Richtung klassischer AST-Parser verschoben werden muss.

Die weitreichendste Änderung in diesem finalen Zyklus war die massive Ausweitung der deterministischen Vorverarbeitung (Preprocessing). Es wurde ein umfassendes Python-Skript integriert, das den Agenten den Großteil der kognitiven Arbeit abnimmt: Es arbeitet auf einer Kopie der Rohdaten, löscht leere Werte ohne Vorbedingungen und bereinigt alte Kommentare. Es verknüpft mehrere Vorbedingungen logisch mit **AND**, korrigiert fehlerhafte Altlogik („Ghost Logic“), markiert unklassifizierte Merkmale und fällt vor allem die strikte, deterministische Entscheidung, ob und wie viele Tabellen für ein Merkmal gebaut werden. Zuletzt wurde eine Hilfsfunktion implementiert, die sicherstellt, dass aus der Agenten-Antwort ausschließlich reines JSON extrahiert wird, um Programmabstürze durch unnötigen Begleittext (z. B. „Hier ist das Ergebnis:“) zu verhindern.

Ergebnis und architektonisches Fazit:

Durch diese Anpassungen entwickelte sich die Architektur von einer reinen KI-Pipeline zu einem *neuro-symbolischen* System. Die Künstliche Intelligenz wurde schrittweise aus

der logischen Entscheidungsfindung verdrängt. Dem LLM verbleibt im finalen Prototyp lediglich die Aufgabe, den vorbereiteten Bauplan in AVC-Code zusammenzusetzen und grundlegend zu validieren.

Die Ergebnisse waren im letzten Testlauf reproduzierbar; das System lieferte bei drei Durchläufen dreimal exakt dasselbe Resultat. Die zentrale Erkenntnis dieser iterativen Entwicklung ist, dass der Problemraum der Code-Migration stark deterministisch ist. Der Einsatz von generativer KI erweist sich für die strukturelle Analyse als ineffizient und ist bei vollständiger Umsetzung mathematischer Solver (wie AST-Parser und SAT-Solver) für diesen Teilprozess weitestgehend obsolet.

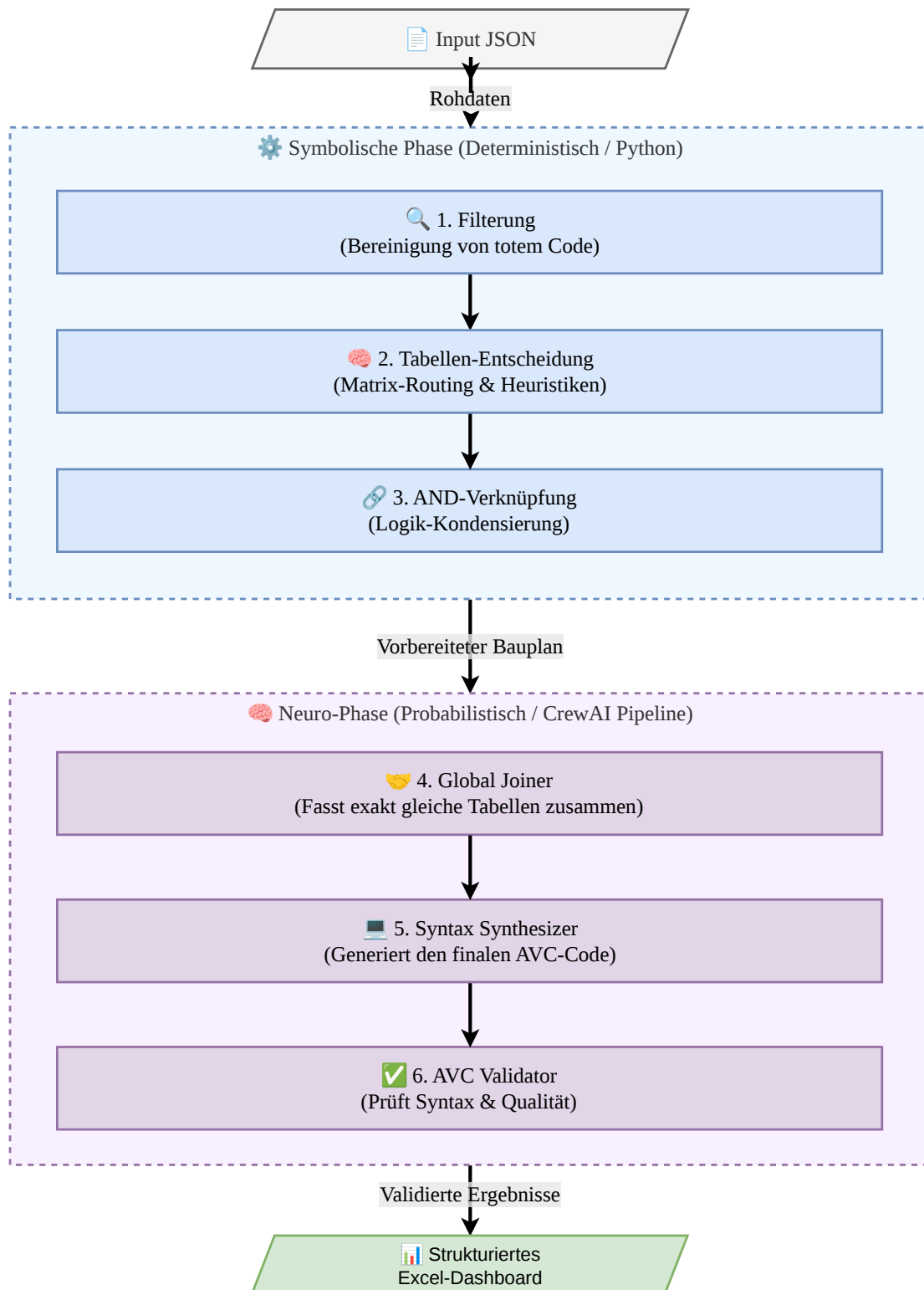


Abbildung 6.2: Finale Gesamtarchitektur des neuro-symbolischen Prototyps (V4).

Kapitel 7

Qualitative Evaluation

7.1 Methodisches Setup und Durchführung

Da innerhalb der Bearbeitungszeit kein Zugriff auf bereits vollständig manuell migrierte Kundenprojekte (*Ground Truth*) bestand, war eine rein quantitative Messung (z. B. Fehlerquoten) nicht durchführbar. Um den Prototyp stattdessen fundiert zu evaluieren, wurde das Forschungsdesign als qualitative Fallstudie gemäß den etablierten Richtlinien für das Software Engineering nach Runeson und Höst strukturiert (Runeson & Höst, 2009). Das Vorgehen definiert sich durch folgende Kernkriterien:

Zielsetzung und Forschungsfragen (Objective & Research Questions):

Das Ziel der Fallstudie ist die qualitative Überprüfung der vom Prototyp getroffenen Architekturentscheidungen und des generierten Codes. Um das System gezielt auf seine Praxistauglichkeit zu testen, wurden drei zentrale Fragestellungen definiert: (1) Sind die Herleitungen der Agenten für einen menschlichen Entwickler nachvollziehbar? (2) Bleibt die fachliche Logik (Semantik) erhalten? (3) Ist der generierte SAP-AVC-Code syntaktisch valide?

Fallauswahl und Selektionsstrategie (The Case & Selection Strategy):

Der untersuchte „Fall“ ist die qualitative Evaluation der System-Ausgaben durch einen Fachexperten der msg systems ag. Als Datengrundlage dienten fünf explizit neue, im Rahmen der Evaluierung vom System prozessierte Merkmale samt ihrer Vorbedingungen aus einem realen Kundenprojekt. Die Stichprobengröße von $n = 5$ wurde bewusst gewählt, um die methodische Sättigung (Theoretical Saturation) zu erreichen. Da die Systemarchitektur auf deterministischen Mustern basiert, wiederholten sich die systemischen Fehlermuster (z.B. bei der Tabellenbildung) bereits nach diesen fünf Testläufen, sodass der Fachexperte bestätigte, dass eine weitere Ausweitung der Stichprobe keine grundlegend neuen Erkenntnisse zur Architektur-Validität mehr geliefert hätte (vgl. Anhang, S. 74). Die Analyse

dieser fünf Fälle ermöglichte somit eine valide Identifikation der systemischen Grenzen und der architektonischen Optimierungspotenziale.

Datenerhebungsmethode (Methods):

Die Datenerhebung erfolgte in einer 60-minütigen Sitzung und kombinierte zwei Methoden. Zunächst wurde das *Think-Aloud*-Verfahren (Methode des lauten Denkens) angewendet (Seaman, 1999). Der Experte verglich die fünf ursprünglichen JSON-Daten völlig frei mit den generierten Dashboards und sprach seine Fehleranalysen und Gedanken laut aus. Ziel war es, das Vorgehen nicht durch vorgegebene Fragen einzuengen, sondern völlig offen für unerwartete Erkenntnisse und das implizite Fachwissen (*Tacit Knowledge*) des Entwicklers zu sein. Der Autor verhielt sich hierbei als rein neutraler Beobachter.

Im direkten Anschluss an diese offene Phase wurde ein semi-strukturiertes Interview geführt (Runeson & Höst, 2009). Anhand eines kurzen Leitfadens wurden die zuvor definierten Kernfragestellungen (Nachvollziehbarkeit, Semantik, Syntax, Praxistauglichkeit) gezielt abgefragt, um die freien Beobachtungen des Experten strukturiert zusammenzufassen. Das vollständige Transkript dieser Sitzung (siehe Anhang) bildet die Datengrundlage für die folgende Auswertung. **Datenauswertung und Codierungsverfahren (Data Analysis):**

Die Auswertung des transkribierten Interviews erfolgte methodisch durch eine strukturierende Inhaltsanalyse nach Mayring. Dies entspricht exakt dem *Template Approach* (vorlagenbasierter Ansatz) für Fallstudien im Software Engineering nach Runeson und Höst (Runeson & Höst, 2009). Dabei wurde ein rein deduktives Vorgehen gewählt, bei dem *a priori* feste Codierungskategorien gebildet wurden, die sich direkt aus den Kernfragestellungen der Evaluierung ableiten: (1) Nachvollziehbarkeit, (2) Semantische Äquivalenz, (3) Syntaktische Korrektheit und (4) Praxistauglichkeit. Im Gegensatz zu der Auswertung des Experteninterviews im Kapitel 4. wurde in dieser Phase bewusst auf einen zusätzlichen induktiven Codierungsdurchgang verzichtet. Da die vier definierten Hauptkategorien inhaltlich weit genug gefasst sind, boten sie ausreichend methodischen Raum, um alle freien Beobachtungen und unerwarteten Erkenntnisse des Experten bei der Code-Review vollständig und verlustfrei aufzunehmen. Das Transkript wurde unter Einsatz der Software QCAmap systematisch analysiert und die relevanten Zitate den jeweiligen Kategorien zugeordnet. Dieses strukturierte Vorgehen stellt eine transparente Beweiskette (*Chain of Evidence*) sicher, sodass die finalen Schlussfolgerungen logisch und nachvollziehbar aus den ursprünglichen Rohdaten hervorgehen (Runeson & Höst, 2009).

7.2 Ergebnisse der Experten-Evaluation

7.2.1 Nachvollziehbarkeit (Explainability & Traceability)

Das Kriterium der Nachvollziehbarkeit bewertet, inwiefern die vom System getroffenen architektonischen Entscheidungen für den menschlichen Fachexperten transparent, lesbar und logisch begründet sind.

Grundsätzlich bewertete der Experte den Ansatz des Agentensystems positiv, Herleitungen strikt vom eigentlichen Code zu trennen und strukturiert zu dokumentieren. Besonders bei simplen Werte-Abbildungen konnte das System durch klare Beschreibungen überzeugen, was der Experte im Interview bestätigte: „Oben steht als Herleitung: Die Entscheidung ist 'NO_TABLE'. Da wird für jeden Eintrag aus dem Mapping ein sauberes IF-Statement erstellt.“ Dieser Detailgrad hilft dem menschlichen Entwickler, die generierte Logik ohne tiefgehende Code-Analyse zu verifizieren.

Trotz dieser positiven Ansätze zeigte die Evaluierung signifikante Schwächen in der Erklärungstiefe auf, sobald die zugrunde liegende Logik der Eingangsdaten komplexer wurde. Der Kern der Kritik lag darin, dass das System zwar korrekte finale Design-Entscheidungen traf, das entscheidende *Warum* jedoch oft unzureichend dokumentierte. Bei komplexen Bedingungen war nicht immer ersichtlich, warum das System sich gegen eine Tabellenstruktur (NO_TABLE) entschied, was zu der Anmerkung führte: „Für mich ist nach wie vor interessant, wie der Architekt zu der Entscheidung TABLE oder NO_TABLE kommt.“

Die Begründungen der KI blieben in diesen Fällen zu oberflächlich. Das System betrachtete die Merkmale oft nur pauschal, anstatt die architektonische Entscheidung granular auf Basis der einzelnen Werte und ihrer spezifischen Vorbedingungen herzuleiten. Um im späteren Praxiseinsatz einen effizienten Review-Prozess zu ermöglichen, muss das Agentensystem seine Zwischenschritte präziser darlegen und genauer pro Wert argumentieren, anstatt die Herleitung wie eine *Blackbox* zu behandeln und lediglich das finale Design-Pattern auszugeben.

7.2.2 Semantische Äquivalenz und Logik (Struktur & Architektur)

Das Kriterium der semantischen Äquivalenz und Logik fokussiert sich auf die inhaltliche Richtigkeit des generierten Codes. Es bewertet, ob die ursprüngliche fachliche Logik der SAP-LO-VC-Modelle korrekt und verlustfrei in die Architektur der neuen AVC-Engine übertragen wurde.

Der Fachexperte lobte in diesem Zusammenhang zunächst die grundsätzliche Zuverlässigkeit des Agentensystems bei der Extraktion von Regeln. Die KI arbeitete präzise mit den übergebenen Daten, was der Experte wie folgt zusammenfasste: „Es ist manuell

schwer zu überprüfen, ob hier in der Logik etwas fehlt, aber mein Eindruck ist: Der Agent arbeitet zuverlässig. Ich sehe hier keine hinzuhalluzinierten Bedingungen, und er scheint auch keine vorhandenen Bedingungen zu übersehen.“

Einen besonders hohen inhaltlichen Mehrwert lieferte das System zudem durch das korrekte, automatische Erkennen von unklassifizierten Merkmalen (*Unclassified Characteristics*). Die KI identifizierte in den meisten Fällen zielsicher, welche Merkmale im Kontext des aktuellen Materials keine Relevanz mehr besaßen („Das hat der Agent quasi richtig gemacht“).

Trotz dieser starken Extraktionsleistung offenbarte die Evaluierung jedoch erhebliche architektonische Lücken bei der anschließenden Transformation der Logik in Variante Tabellen. Diese ließen sich auf drei primäre Schwachstellen zurückführen:

1. Fehlendes Wildcard-Konzept (Platzhalter):

Das System scheiterte systematisch daran, *Wildcards* in die Variantentabellen zu integrieren. „Ich habe in allen generierten Dateien noch keine einzige Wildcard gesehen. Dieses Konzept ist dem Modell bisher noch nicht bekannt“, merkte der Experte an. Da das Agentensystem nicht anwendete, wie es Lücken in der Tabellenmatrix mit Platzhaltern füllen konnte, wick es bei komplexen Bedingungen fälschlicherweise auf die Entscheidung `NO_TABLE` (Einzel-Constraints) aus, obwohl zwingend eine Tabelle hätte aufgebaut werden müssen.

2. Falscher Umgang mit unklassifizierten und bedingungslosen Werten:

Obwohl die KI unklassifizierte Merkmale korrekt erkannte, zog sie daraus nicht konsequent die richtige architektonische Schlussfolgerung. Anstatt diese Merkmale vollständig aus dem Tabellenaufbau zu streichen und Tabellen nur noch im Kontext der tatsächlich klassifizierten Merkmale aufzuspannen, führte sie diese teilweise weiter, wodurch der Tabellenaufbau logisch kollabierte. Eng damit verknüpft war das Ignorieren von Werten ohne explizite Vorbedingung. Das System übersah dabei eine fundamentale Logikregel der Konfiguration: „Ein Wert ohne Bedingung ist immer gültig. Ergo muss der Wert in die Tabelle aufgenommen und alle anderen Bedingungsspalten in dieser Zeile mit einer Wildcard versehen werden.“

3. Redundante Tabellen und falsche Schnittmengen-Logik:

Ein weiterer Kritikpunkt war die Erzeugung fehlerhafter Schnittmengen. Zwar lobte der Experte den Versuch der KI, bei sehr komplexen Regelbrüchen die Logik auf zwei separate Tabellen aufzuteilen, wies jedoch auf einen kritischen Denkfehler hin: Wenn in der AVC-Engine zwei Tabellen für dasselbe Merkmal aufgerufen werden, gilt als erlaubter Wertevorrat ausschließlich die exakte Schnittmenge beider Tabellen. Die KI erzeugte dadurch „Gegenspieler-Bedingungen“, bei denen am Ende fälschlicherweise kein gültiger Wert mehr übrig blieb. Stattdessen hätte die KI deckungsgleiche Logiken erkennen, in einer einzigen Tabelle zusammenführen („verheiraten“) und mit Wildcards auffüllen müssen.

7.2.3 Syntaktische Korrektheit (AVC-Compliance)

Das Kriterium der syntaktischen Korrektheit überprüft, ob der vom Agentensystem generierte Quellcode den strengen formalen Vorgaben der neuen *Advanced Variant Configuration* (AVC) in SAP entspricht. Während die vorangegangenen Kriterien die inhaltliche Logik bewerteten, geht es hierbei um formale Syntaxfehler, die als „Hard-Fails“ dazu führen würden, dass der Code vom SAP-System abgewiesen und nicht ausgeführt werden kann.

Aus der Evaluation ging hervor, dass das System zwar die Grundmatrix der Tabellen oftmals richtig anlegte, bei der exakten Formulierung der Syntax-Strings jedoch wiederkehrende Fehler produzierte. Diese Fehler lassen sich im Wesentlichen in zwei Kategorien unterteilen:

1. Falsche Formatierung von Tabellenaufrufen:

Obwohl die inhaltliche Struktur einer Variantentabelle teilweise korrekt hergeleitet wurde, scheiterte das System mehrfach an der exakten Syntax, die benötigt wird, um diese Tabelle im Constraint-Code aufzurufen. Der Experte wies bei der Durchsicht der generierten Artefakte mehrfach darauf hin, dass die Aufruf-Syntax fehlerhaft war („Die Syntax unten, um die Tabelle aufzurufen, ist zwar noch falsch formatiert“ sowie „Die Syntax des Aufrufs ist wieder falsch“). Derartige Formatierungsfehler zeigen, dass das *Large Language Model* die strikten syntaktischen Schablonen für Tabellenaufrufe in AVC noch nicht verinnerlicht hat und hier eine präzisere Steuerung oder Nachbearbeitung erforderlich ist.

2. Verwendung unzulässiger AVC-Begrifflichkeiten:

Ein noch kritischerer Fehler offenbarte sich bei der Generierung von negierenden Bedingungen. Das System nutzte bei der Code-Erzeugung den Ausdruck `NOT SPECIFIED`, um zu definieren, dass ein bestimmtes Merkmal nicht bewertet wurde. Der Experte identifizierte dies als fundamentalen Syntaxfehler für die Zielarchitektur: „Das Problem ist: `NOT SPECIFIED` ist in der neuen AVC-Engine kein zulässiger Begriff in Constraints.“ Während der Begriff in der alten LO-VC-Welt teilweise Anwendung fand, führt er in der modernen AVC-Umgebung zwingend zu einem Kompilierungsfehler.

Zusammenfassend lässt sich festhalten, dass das Agentensystem auf syntaktischer Ebene noch zu unpräzise arbeitet. Um den generierten Code direkt und ohne manuelle syntaktische Korrekturen (*Debugging*) in ein SAP-System migrieren zu können, muss dem Modell das strikte, AVC-spezifische Regelwerk künftig durch engmaschigere Prompt-Restriktionen oder syntaktische Post-Processing-Schritte noch deutlich härter vorgegeben werden.

7.2.4 Praxistauglichkeit und Business Value

Das abschließende Kriterium der Praxistauglichkeit bewertet den direkten geschäftlichen Mehrwert (*Business Value*) des Agentensystems. Es beantwortet die Frage, ob der aktuelle

Prototyp im industriellen Alltag der *msg systems ag* bereits eingesetzt werden kann, um den manuellen Aufwand und die Zeit bei Code-Migrationen messbar zu reduzieren.

Bezogen auf die exakte, vorliegende Version des Prototyps (V1) zog der Experte ein ernüchterndes, aber klares Fazit: „Das, was am Ende als Code herausgekommen ist, wäre so auf alle Fälle noch nicht direkt im System produktiv nutzbar, eine Nacharbeit durch den Entwickler ist aktuell noch zwingend notwendig.“ Auf die explizite Rückfrage, ob das Tool in seiner aktuellen Form bereits Migrationszeit einsparen würde, verneinte der Fachexperte dies, da der Aufwand, die architektonischen Fehler (wie fehlende Wildcards oder redundante Tabellen) manuell im Quellcode zu korrigieren, den Zeitgewinn der automatischen Extraktion noch zunichtemachen würde.

Dennoch sprach der Experte dem System das grundlegende Potenzial nicht ab, sondern identifizierte bereits im jetzigen Entwicklungsstadium einen signifikanten Teilerfolg: „Was aber definitiv einen massiven inhaltlichen Mehrwert bietet, ist, dass das System vollautomatisch erkannt hat, welche Merkmale überhaupt nicht klassifiziert sind. Das hilft dem menschlichen Bearbeiter bereits enorm.“

Der Weg zu einem produktiv nutzbaren und zeiteinsparenden Werkzeug ist laut dem Experten klar umrissen und erfordert keine konzeptionelle Neuentwicklung, sondern vielmehr ein gezieltes Nachschärfen der architektonischen Schlussfolgerungen. Die unmittelbaren *To-Dos* für nachfolgende Iterationen umfassen (1) das konsequente Löschen unklassifizierter Merkmale aus den Tabellendefinitionen und (2) die Einführung eines Wildcard-Konzepts (Platzhalter) zum Auffüllen von Werten ohne Vorbedingung.

Der Experte schloss die Evaluation mit einer sehr positiven Prognose für den zukünftigen Business Value ab: „Wenn das Tool den korrekten Umgang mit diesen nicht klassifizierten Merkmalen lernt und den Zusatz bekommt, daraus komplette, große Tabellen mit Wildcards aufzubauen, dann wäre es auf jeden Fall eine enorme Hilfe für den Entwickler und man wäre deutlich schneller am Ziel.“ Das Agentensystem besitzt somit – nach Behebung der identifizierten Architekturschwächen – ein hohes Potenzial, die Migration von LO-VC nach AVC in der Praxis erheblich zu beschleunigen.

7.3 Limitierungen der Fallstudie (Threats to Validity)

Jede empirische Untersuchung unterliegt gewissen Limitierungen, die die Aussagekraft der Ergebnisse beeinflussen können. Gemäß den Richtlinien für Fallstudien im *Software Engineering* nach Runeson und Höst werden im Folgenden die potenziellen Gefahren für die Validität (*Threats to Validity*) dieser Evaluierung kritisch reflektiert (Runeson & Höst, 2009).

Externe Validität (Generalisierbarkeit):

Die größte Limitierung dieser Evaluierung liegt in der eingeschränkten externen Validität. Die Fallstudie wurde mit lediglich einem einzelnen Fachexperten ($n = 1$) und anhand

von fünf ausgewählten Materialmerkmalen aus einem spezifischen Kundenprojekt der *msg systems ag* durchgeführt. Eine statistische Generalisierung auf alle denkbaren SAP-Migrationsprojekte ist somit nicht möglich. Dennoch erlaubt das qualitative Design eine analytische Generalisierung Runeson und Höst (2009): Da die architektonischen Schwächen (wie das fehlende Wildcard-Konzept) auf der systemischen Ebene des LLMs und nicht auf branchenspezifischen Daten beruhen, ist davon auszugehen, dass diese Problematiken auch in anderen Projektkontexten in ähnlicher Form auftreten würden.

Reliabilität (Zuverlässigkeit der Daten):

Die Reliabilität bewertet, inwiefern ein anderer Forscher zu den gleichen Ergebnissen gekommen wäre (Runeson & Höst, 2009). Da die Codierung und Auswertung des Experteninterviews ausschließlich durch den Autor dieser Arbeit stattfand, besteht das Risiko eines unbewussten *Researcher Bias* (Forschervoreingenommenheit). Um dieses Risiko zu minimieren, wurde das deduktive *Template Approach*-Verfahren nach Mayring angewendet (Runeson & Höst, 2009). Durch die feste *a priori*-Definition der vier Evaluationskategorien wurde der Interpretationsspielraum bei der Zuordnung der Zitate stark eingegrenzt. Zudem sorgt die vollständige Bereitstellung des Transkripts im Anhang für maximale Transparenz und Nachvollziehbarkeit der Beweiskette (*Chain of Evidence*) (Runeson & Höst, 2009).

Konstruktvalidität (Construct Validity):

Die Konstruktvalidität hinterfragt, ob die gewählte Methodik tatsächlich das misst, was sie messen soll (Runeson & Höst, 2009). Ein potenzielles Risiko bestand darin, dass der Experte seine Aussagen aufgrund der Anwesenheit des Autors unbewusst anpassen oder Kritik zurückhalten könnte (*Hawthorne-Effekt*). Um dem entgegenzuwirken, wurde bewusst die Methode des *Think-Aloud* gewählt, bei der der Experte völlig frei und un gelenkt seine direkten Gedanken verbalisierte. Der Autor verhielt sich strikt als stiller Beobachter und griff nicht rechtfertigend in den Analyseprozess ein, wodurch eine möglichst objektive und ehrliche Bewertung der Artefakte sichergestellt wurde.

Interne Validität:

Die interne Validität ist primär für erklärende (*explanatory*) Studien relevant, die kausale Zusammenhänge untersuchen (Runeson & Höst, 2009). Da das Ziel dieser Fallstudie rein explorativ und beschreibend war (die Identifikation von Architekturfehlern im generierten Code), spielt dieser Validitätsaspekt für die vorliegende Evaluation eine untergeordnete Rolle (Runeson & Höst, 2009).

Kapitel 8

Fazit und Ausblick

8.1 Zusammenfassung der Arbeit

Die vorliegende Masterarbeit befasste sich mit der hochaktuellen und komplexen Herausforderung produzierender Unternehmen, historisch gewachsene Produktmodelle der SAP-Variantenkonfiguration von der imperativen LO-VC-Logik in die moderne, deklarative Advanced Variant Configuration (AVC) zu migrieren[cite: 1]. Da direkte, regelbasierte 1:1-Übersetzer an dem fundamentalen Paradigmenwechsel der Systeme scheitern und monolithische LLM-Ansätze aufgrund der algorithmischen Komplexität zu Halluzinationen neigen, wurde die Konzeption eines Multi-Agenten-Systems (MAS) als innovativer Lösungsansatz gewählt[cite: 1].

Unter Anwendung des methodischen Paradigmas des Design Science Research (DSR) wurde zunächst durch Experteninterviews und einen praktischen Selbstversuch die technologische sowie organisationale Problemstellung durchdrungen[cite: 1]. Dies ermöglichte die Extraktion eines dreistufigen kognitiven Modells (Bereinigung, Architektur-Entwurf, Synthese), welches als Blaupause für das IT-Artefakt diente[cite: 1]. Die anschließende Prototypenentwicklung mit dem Framework CrewAI auf dem SAP AI Core zeigte in iterativen Zyklen, dass rein agentenbasierte Architekturen bei deterministischen Tabellenstrukturen an ihre Grenzen stoßen[cite: 1]. Infolgedessen wurde das System zu einer neuro-symbolischen Architektur weiterentwickelt, bei der deterministische Python-Skripte die komplexe Vorverarbeitung übernehmen und das Sprachmodell gezielt für die Synthese des AVC-Codes eingesetzt wird[cite: 1]. Die abschließende qualitative Evaluation mit einem Fachexperten bestätigte das immense Potenzial des Systems, deckte jedoch auch noch bestehende architektonische Schwächen auf, die einen sofortigen, vollautonomen Produktiveinsatz aktuell noch verhindern[cite: 1].

8.2 Beantwortung der Forschungsfragen

Die im Rahmen der Einleitung definierten Forschungsfragen lassen sich basierend auf den empirischen und praktischen Ergebnissen dieser Arbeit wie folgt beantworten:

Beantwortung der Hauptforschungsfrage:

Inwiefern lässt sich die semantische und syntaktische Migration von historischer LO-VC-Logik zu deklarativer AVC-Syntax durch eine Multi-Agenten-Architektur automatisieren, und wo liegen die systemischen Grenzen dieses Ansatzes?

Die Automatisierung der Migration durch eine reine Multi-Agenten-Architektur ist in der Praxis nur partiell umsetzbar und erfordert zwingend eine methodische Weiterentwicklung hin zu einem neuro-symbolischen System. Während Sprachmodelle (LLMs) eine enorme Stärke bei der semantischen Analyse von historischem Code und der fehlerfreien Identifikation von kognitiven Altlasten aufweisen, liegen ihre systemischen Grenzen in der probabilistischen Verarbeitung von deterministischen SAP-Constraint-Netzwerken. Der KI-basierte Ansatz scheitert insbesondere an der konsistenten Erzeugung fehlerfreier Variantentabellen, dem korrekten Umgang mit Schnittmengen und der Validierung formaler Syntax. Erst durch die gezielte Auslagerung mathematisch-logischer Vorverarbeitungsschritte an deterministische Python-Skripte kann das Potenzial der generativen KI für die finale Code-Synthese praxistauglich und sicher genutzt werden.

Zur detaillierten Aufschlüsselung werden die vier Unter-Forschungsfragen wie folgt beantwortet:

- **FF1 (Problemraum & Baseline):** Welche prozessualen und kognitiven Hürden prägen die manuelle Migration historischer SAP-Modelle in die neue Zielarchitektur, und wie lässt sich die daraus resultierende Baseline zeitlich abschätzen?
 - Die manuelle Migration eines durchschnittlichen Konfigurationsmodells nimmt in der industriellen Praxis laut Expertenschätzung etwa drei bis vier Personentage in Anspruch.
 - Zur zeitlichen Einordnung der Baseline muss zwischen zwei Szenarien unterschieden werden: Bei einem kompletten Neuaufbau (Greenfield-Ansatz) verteilt sich der Aufwand auf etwa 20% für die Merkmalswelt, 40% für die vorbereitende Analyse und Bereinigung von historischem „totem Code“ sowie 40% für die eigentliche syntaktische Umsetzung.
 - Da ein Greenfield-Ansatz maschinell nicht abgebildet werden kann, fokussiert sich der Prototyp auf die *direkte Migration*. In diesem Szenario bildet die prozessuale Transformation der Vorbedingungswelt in deklarative Constraints den absoluten Flaschenhals, welcher rund 66% (zwei Drittel) des Gesamtaufwands ausmacht und somit die relevante zeitliche Zielgröße für das entwickelte Automatisierungswerkzeug definiert.

- **FF2 (Kognitive Modellierung):** Wie lassen sich die manuellen Übersetzungsschritte eines menschlichen Entwicklers methodisch dekonstruieren, um daraus ein geeignetes Rollen- und Aufgabendesign für ein Multi-Agenten-System abzuleiten?
 - Durch einen empirischen Selbstversuch auf Code-Ebene konnte der Übersetzungsprozess erfolgreich in ein dreistufiges, kognitives Phasenmodell dekonstruiert werden.
 - Diese Phasen umfassen: 1. die semantische Bereinigung von Altlasten, 2. die architektonische Entscheidung über Tabellenstrukturen und 3. die finale syntaktische Codierung.
 - Dieses sequenzielle Denkmodell diente als direkte Blaupause für das initiale Rollendesign (Analyst, Architekt, Synthesizer) und die Aufgaben-Delegation innerhalb des CrewAI-Frameworks.
- **FF3 (Architekturentwurf & Limitierungen):** Welche technologischen Herausforderungen ergeben sich beim Einsatz generativer KI-Agenten für komplexe Code-Transformationen, und wie muss die Systemarchitektur iterativ gestaltet werden, um deterministische Zielstrukturen (wie Variantentabellen) zuverlässig zu erzeugen?
 - Die iterative DSR-Entwicklung deckte auf, dass rein generative Agenten bei der Erstellung von Variantentabellen stark halluzinieren, unlogische Spalten erfinden und an der redundanzfreien Definition von Schnittmengen scheitern.
 - Um deterministische Zielstrukturen zuverlässig zu generieren, musste die Systemarchitektur im vierten Iterationszyklus fundamental zu einem neuro-symbolischen Ansatz umgebaut werden.
 - Komplexe Strukturierungsentscheidungen (Analysten- und Architekten-Perspektive) wurden in deterministische Python-Algorithmen ausgelagert, sodass sich die KI ausschließlich auf die exakte Übersetzung der vorbereiteten Logik fokussieren kann.
- **FF4 (Evaluation & Mehrwert):** Welches funktionale Potential und qualitativen Mehrwert liefert das entwickelte Architektur-Artefakt im direkten Vergleich zur manuellen Baseline, und welche funktionalen Grenzen bestehen für einen vollautonomen Praxiseinsatz?
 - Eine direkte, quantitative Zeitersparnis ist durch den aktuellen Prototyp noch nicht messbar, da architektonische Limitierungen (insb. fehlende Wildcard-Logik) noch manuelle Nacharbeiten am Zielcode erforderlich machen.
 - Das System liefert jedoch bereits einen messbaren qualitativen Mehrwert (Business Value): Die automatisierte Identifikation unklassifizierter Merkmale und

- „toter“ Logikbausteine entlastet den Entwickler von rein administrativen Analysetätigkeiten, die in der Praxis oft unterschätzt werden.
- Die Evaluation verdeutlicht, dass die zukünftige Architektur eine hybride Rollenverteilung verfolgen muss: Während deterministische Algorithmen (Python-Skripte/Parser) die strukturelle Transformation (Tabellenbau) übernehmen sollten, dient die KI primär als „beratendes Werkzeug“ auf der „höheren Flugebene“ (gemäß Empfehlung des Fachexperten, vgl. Anhang, S. 74), um Design-Entscheidungen zu validieren und Modell-Optimierungen aufzuzeigen.

8.3 Limitierungen der Arbeit

Die in dieser Arbeit durchgeführte Prototypenentwicklung und Evaluation unterliegt bestimmten methodischen und praktischen Limitierungen, die bei der Interpretation der Ergebnisse berücksichtigt werden müssen.

8.3.1 Eingeschränkte Datenverfügbarkeit und fehlende Ground Truth

Im Umfeld proprietärer SAP-Migrationsprojekte unterliegen historische Konfigurationsdaten strengen Geheimhaltungsrichtlinien. Das Fehlen großflächiger, historischer Altdaten („Ground Truth“) von bereits erfolgreich und manuell migrierten Projekten machte eine statistisch signifikante, quantitative Evaluation (beispielsweise die Messung von Kompilieraten über hunderte Modelle hinweg) im Rahmen dieser Arbeit unmöglich. Aus diesem Grund wurde das Forschungsdesign bewusst und fundiert auf eine tiefgehende qualitative Fallstudie (mit einer Stichprobe von $n = 5$ Merkmalen) fokussiert.

8.3.2 Domänenkomplexität und Expertenverfügbarkeit

Die Logik der SAP-Variantenkonfiguration erfordert massives, oft undokumentiertes Domänenwissen. Der beschränkte Zugriff auf Expertenressourcen im Tagesgeschäft der Unternehmensberatung führte während der iterativen Prototypenentwicklung partiell zu konzeptionellen Irrwegen. Ein Beispiel hierfür war die initiale Annahme, dass Werte ohne Vorbedingungen ignoriert werden könnten, was sich in der Evaluation als fehlendes Wildcard-Konzept herausstellte. Diese Herausforderungen beim Wissensaufbau mindern jedoch nicht den Wert der Ergebnisse, sondern spiegeln vielmehr die reale Komplexität und den steilen Lernprozess industrieller Migrationsprojekte wider.

8.4 Gelernte Lektionen und zukünftige Forschungsfelder

8.4.1 Wissenschaftliche Einordnung: Warum Künstliche Intelligenz?

Kritisch betrachtet wirft die im letzten Zyklus vorgenommene Auslagerung des architektonischen Tabellenbaus an deterministische Python-Skripte die berechtigte Frage auf, warum überhaupt der Einsatz generativer KI erforscht wurde, wenn klassische, deterministische Ansätze an dieser Stelle überlegen scheinen. Genau dieses Erkenntnis bildet jedoch den stärksten wissenschaftlichen Beitrag dieser Arbeit. Entgegen dem aktuellen branchenweiten Hype, dass Large Language Models jegliche Form von Code-Migration vollautonom bewältigen können, belegt dieser Prototyp empirisch die systemischen Grenzen probabilistischer KI bei domänenspezifischen, stark vernetzten Constraint-Problemen. Die Ergebnisse verdeutlichen, dass nicht die vollständige Ablösung durch KI, sondern die orchestrierte Symbiose – das neuro-symbolische Paradigma aus deterministischen Skripten für harte Logik und generativer KI für semantische Analyse und Code-Synthese – den zukunftsfähigsten Ansatz darstellt.

8.4.2 Technische Weiterentwicklung des Artefakts

Um den Prototyp von einem Proof-of-Concept zu einem produktiv einsetzbaren Werkzeug in der SAP-Migrationsberatung weiterzuentwickeln, bedarf es zukünftig gezielter informationstechnischer Erweiterungen. Aus der qualitativen Evaluation lassen sich drei primäre Handlungsfelder ableiten:

- **Integration lokaler Validierung zur Simulation fehlenden Compiler-Feedbacks (AST-Parser):** Wie in den theoretischen Grundlagen hergeleitet, stellt das fehlende Feedback einer Testumgebung eine zentrale Fehlerquelle für Code-generierende Sprachmodelle dar. Da proprietäre SAP-Systeme keinen offenen Compiler-Zugang zur Laufzeit bieten, musste im aktuellen Prototyp auf eine rein textbasierte Syntax-Prüfung durch einen KI-Agenten zurückgegriffen werden, welche sich in der Evaluation als unzureichend erwiesen hat. Zukünftige Iterationen sollten daher zwingend einen lokalen *Abstract Syntax Tree (AST)* Parser integrieren. Dieser kann das fehlende System-Feedback deterministisch simulieren, um den generierten Constraint-Code vor der Übertragung auf formale AVC-Regelbrüche (wie die fehlerhafte Nutzung des Begriffs „NOT SPECIFIED“) zu prüfen und den KI-Agenten in eine echte Korrekturschleife zu zwingen.
- **Implementierung von SAT-Solvern:** Das Kernproblem der redundanten und logisch überlappenden Tabellen konnte durch Sprachmodelle nicht verlässlich gelöst

werden[cite: 1]. Die Anbindung eines mathematischen SAT-Solvers an die neuro-symbolische Pipeline könnte künftig deterministisch garantieren, dass nur logisch disjunkte und redundanzfreie Tabellenkonstrukte an das SAP-System übergeben werden[cite: 1].

- **Erweiterung um ein holistisches Wildcard-Konzept:** Wie die Experten-Evaluation aufzeigte, scheitert der Code aktuell am Umgang mit bedingungslosen Werten[cite: 1]. Das System muss zwingend dahingehend weiterentwickelt werden, dass leere Parameter-Zellen in Variantentabellen systematisch mit Platzhaltern (Wildcards) aufgefüllt werden, um korrekte Schnittmengen zu gewährleisten[cite: 1].

8.4.3 Reevaluation der Modellwahl: Code-LLMs für die reine Code-Synthese

Ein weiteres zentrales Ergebnis aus dem iterativen Entwicklungsprozess betrifft die Wahl des zugrundeliegenden Sprachmodells. In den theoretischen Grundlagen wurde basierend auf dem aktuellen Stand der Forschung die Annahme getroffen, dass für die Code-Migration und den damit verbundenen Paradigmenwechsel große, allgemeine Sprachmodelle erforderlich sind, da spezialisierte Code-Modelle bei komplexen Übersetzungsaufgaben oft an mangelnden Reasoning-Fähigkeiten scheitern. Durch die evolutionäre Weiterentwicklung zur neuro-symbolischen Architektur in Zyklus 4 hat sich das tatsächliche Aufgabenspektrum der KI innerhalb des Systems jedoch fundamental gewandelt. Da die komplexe architektonische Entscheidungsfindung (wie das Bündeln von Vorbedingungen und der Tabellenbau) nun nahezu vollständig durch deterministische Python-Skripte abgedeckt wird, beschränkt sich die Aufgabe des verbleibenden KI-Agenten primär auf die syntaktische Code-Generierung auf Basis eines festen Bauplans. Für zukünftige Entwicklungen ergibt sich daraus die klare Empfehlung, die anfängliche Modellwahl zu reevaluieren. Da die KI nicht länger tiefgreifend abstrahieren muss, sollte getestet werden, ob kleinere, stark auf Quellcode spezialisierte Modelle (Code-LLMs) für den finalen Synthesizer-Agenten effizienter eingesetzt werden können. Diese Modelle könnten die verbleibende, eng umrissene Programmieraufgabe potenziell schneller und kostengünstiger lösen als ressourcenintensive Allzweckmodelle.

8.5 Schlusswort

Die vorliegende Arbeit verdeutlicht eindrucksvoll die Potenziale, aber auch die Limitationen generativer Künstlicher Intelligenz im hochspezialisierten Software Engineering[cite: 1]. Die automatisierte Systemmigration von imperativen zu deklarativen Architekturen ist kein rein linguistisches Übersetzungsproblem, sondern erfordert ein tiefes algorithmisches

und logisches Verständnis[cite: 1]. Der entwickelte neuro-symbolische Ansatz beweist, dass die Zukunft komplexer Code-Transformationen nicht in monolithischen KI-Modellen liegt, sondern in der intelligenten Orchestrierung von Large Language Models mit harten, deterministischen Algorithmen[cite: 1]. Wenn die identifizierten Schwachstellen im Bereich der Syntax-Validierung und Tabellenbildung in zukünftigen Iterationen behoben werden, bietet das konzipierte System das Fundament, um einen der ressourcenintensivsten Flaschenhälse der industriellen S/4HANA-Transformation nachhaltig zu automatisieren[cite: 1].

Literaturverzeichnis

- Blumöhr, U., Kölbl, A., Neuhaus, M., & Ukalovic, M. (2023). *Advanced Variant Configuration in SAP S/4HANA*. Rheinwerk Publishing. Verfügbar 6. Mai 2026 unter <https://katalog.ub.uni-heidelberg.de/titel/69115224>
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., & Askell, A. (2020). Language Models Are Few-Shot Learners. *Advances in neural information processing systems*, 33, 1877–1901. Verfügbar 16. Mai 2026 unter https://proceedings.neurips.cc/paper_files/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html?utm_source=transaction&utm_medium=email&utm_campaign=linkedin_newsletter
- Chase, H. (2024). *Langchain*. <https://github.com/langchain-ai/langchain>
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021, 14. Juli). *Evaluating Large Language Models Trained on Code*. arXiv: 2107.03374 [cs]. <https://doi.org/10.48550/arXiv.2107.03374>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. *MIS quarterly*, 28(1), 75–106. Verfügbar 20. Juni 2026 unter <https://misq.umn.edu/misq/article/28/1/75/261>
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Yau, S., Lin, Z., & Zhou, L. (2024). MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *International Conference on Learning Representations, 2024*, 23247–23275. Verfügbar 20. Mai 2026 unter https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html
- Hove, S. E., & Anda, B. (2005). Experiences from Conducting Semi-Structured Interviews in Empirical Software Engineering Research. *11th IEEE International Software Metrics Symposium (METRICS'05)*, 10–pp. Verfügbar 31. März 2026 unter <https://ieeexplore.ieee.org/abstract/document/1509301/>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). Swe-Bench: Can Language Models Resolve Real-World Github Issues? *International Conference on Learning Representations, 2024*, 54107–54157. Verfügbar

20. Mai 2026 unter https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html
- Karabiyik, M. A. (2025). RefactorGPT: A ChatGPT-based Multi-Agent Framework for Automated Code Refactoring. *PeerJ Computer Science*, 11, e3257. Verfügbar 21. April 2026 unter <https://peerj.com/articles/cs-3257/>
- Li, G., Hammoud, H., Itani, H., Khizbullin, D., & Ghanem, B. (2023). Camel: Communicative Agents forMMindExploration of Large Language Model Society. *Advances in neural information processing systems*, 36, 51991–52008. Verfügbar 20. Mai 2026 unter <https://proceedings.neurips.cc/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html>
- Liu, D., Upadhyay, K., Chhetri, V., Siddique, A. B., & Farooq, U. (2025). A Large-Scale Study on the Development and Issues of Multi-Agent AI Systems. *2025 IEEE International Conference on Big Data (BigData)*, 7785–7792. Verfügbar 2. Juni 2026 unter <https://ieeexplore.ieee.org/abstract/document/11402104/>
- Matilainen, A. (2024). Change Requirements on Product Data Structures in S/4HANA Implementation. Verfügbar 21. April 2026 unter <https://osuva.uwasa.fi/items/8bf7ae52-e756-4fca-9a25-c0510c58ad34>
- Mayring, P. (2014). Qualitative Content Analysis: Theoretical Foundation, Basic Procedures and Software Solution. Verfügbar 7. April 2026 unter https://www.ssoar.info/ssoar/bitstream/handle/document/39517/ssoar-2014-mayring-Qualitative_content_analysis_theoretical_foundation.pdf
- Moti, Z., Soudani, H., & van der Kogel, J. (2026, 14. März). *LegacyTranslate: LLM-based Multi-Agent Method for Legacy Code Translation*. arXiv: 2603.14054 [cs]. <https://doi.org/10.48550/arXiv.2603.14054>
- Oueslati, K., Lamothe, M., & Khomh, F. (2026, 4. März). *RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring*. arXiv: 2511.03153 [cs]. <https://doi.org/10.48550/arXiv.2511.03153>
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., & Cong, X. (2024). Chatdev: Communicative Agents for Software Development. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 15174–15186. Verfügbar 20. Mai 2026 unter <https://aclanthology.org/2024.acl-long.810/>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Russell, S. J. (2010). *Artificial Intelligence a Modern Approach*. Pearson Education, Inc. Verfügbar 26. Mai 2026 unter <https://scholar.alaqsa.edu.ps/9195/1/Artificial%20Intelligence%20A%20Modern%20Approach%20%283rd%20Edition%29.pdf%20%28%20PDFDrive%20%29.pdf>

- Seaman, C. B. (1999). Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Transactions on software engineering*, 25(4), 557–572. Verfügbar 31. März 2026 unter <https://ieeexplore.ieee.org/abstract/document/799955/>
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in neural information processing systems*, 36, 8634–8652. Verfügbar 20. Mai 2026 unter https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in neural information processing systems*, 30. Verfügbar 16. Mai 2026 unter <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., & Liu, J. (2024). Autogen: Enabling next-Gen LLM Applications via Multi-Agent Conversations. *First Conference on Language Modeling*. Verfügbar 20. Mai 2026 unter https://openreview.net/forum?id=BAakY1hNKS&utm_source=
- Xu, Y., Lin, F., Yang, J., Tse-Hsun, Chen & Tsantalis, N. (2025, 27. März). *MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration*. arXiv: 2503.14340 [cs]. <https://doi.org/10.48550/arXiv.2503.14340>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *React: Synergizing Reasoning and Acting in Language Models*. Verfügbar 20. Mai 2026 unter <https://arxiv.org/abs/2210.03629>
- Zheng, Z., Ning, K., Wang, Y., Zhang, J., Zheng, D., Ye, M., & Chen, J. (2023). *A Survey of Large Language Models for Code: Evolution, Benchmarking, and Future Trends*. Verfügbar 14. Mai 2026 unter <https://arxiv.org/abs/2311.10372>

Anhang A

Interview- und Evaluationsleitfaden

A.1 Leitfaden Experteninterview

Interview-Ziele für FF1:

1. Definition des exakten manuellen Prozessablaufs („Was macht der Mensch“).
2. Quantifizierung des zeitlichen Aufwands („Wie lange dauert das“).
3. Identifikation des kognitiven Aufwands und der Fehlerquellen („Was ist daran so schwer“).

Phase 0: Einleitung & Warm-up (ca. 5 Minuten)

(Regieanweisung: Ziel dieser Phase ist es, den ethischen Rahmen zu klären, Vertrauen aufzubauen und den Experten in einen erzählenden Modus zu bringen.)

Begrüßung & Kontext:

„Vielen Dank für deine Zeit. Das Ziel meiner Masterarbeit ist es, ein Multi-Agenten-System zu bauen, das die Migration von LO-VC nach AVC automatisiert. Um zu beweisen, dass dieses System eine echte Erleichterung bringt, muss ich zunächst die ‘menschliche Baseline’ verstehen: Wie läuft die manuelle Migration aktuell ab, wie lange dauert sie und wo liegen die Hürden“

Formalia (Ethik & Transkription):

„Bevor wir starten, kurz zum formellen Ablauf: Um keine Details zu verpassen, würde ich das Gespräch gerne als Audio aufzeichnen. Die Aufnahme wird im Nachgang für die Auswertung wörtlich transkribiert. Das Transkript schicke ich dir gerne zur Freigabe zu. In meiner Masterarbeit werden alle Aussagen anonymisiert verwendet, sodass keine Rückschlüsse auf dich persönlich oder spezifische Kundenprojekte gezogen werden können. Bist du damit einverstanden“

Eisbrecher-Frage:

Hauptfrage 1: „Zum Start: Kannst du kurz deine aktuelle Rolle beschreiben und in welchem Umfang du in deinem Projektalltag mit LO-VC und AVC-Code zu tun hast?“

Phase 1: Der manuelle Prozessablauf (ca. 10 Minuten)

(Regieanweisung: Trichter-Modell anwenden. Den Experten den Prozess frei beschreiben lassen, nicht unterbrechen. Erst danach anhand der Probes gezielt Lücken schließen, falls die in den Probes erwähnten Fragen nicht bereits beantwortet wurden.)

- **Hauptfrage 2 (Offen):** „Stell dir vor, du hast eine typische, komplexe LO-VC Prozedur vor dir, die beispielsweise harte Berechnungen und weiche Vorschlagswerte mischt. Kannst du mich Schritt für Schritt durch deinen Arbeitsablauf führen, wie du diese nach AVC migrierst?“
 - **Probe A (Werkzeuge):** „Nutzt du dafür bestimmte Tools oder machst du das direkt im SAP-System?“
 - **Probe B (Splitting):** „Nach welchen Kriterien entscheidest du, welche Zeilen in eine neue Prozedur wandern und was ein Constraint wird?“
 - **Probe C (Legacy-Funktionen):** „Wie gehst du mit veralteten Aufrufen wie den SAP_VF-Funktionen um?“

Phase 2: Zeitlicher Aufwand & Metriken (ca. 10 Minuten)

(Regieanweisung: Ziel ist es, harte Metriken für die Baseline zu sammeln. Vage Aussagen wie ‘lange’ müssen auf konkrete Zeiten festgenagelt werden.)

- **Hauptfrage 3 (Offen):** „Wie würdest du den zeitlichen Aufwand für diese manuellen Migrationsschritte in euren Projekten beschreiben?“
 - **Probe A (Durchschnitt):** „Können wir das in Minuten oder Stunden fassen? Wie lange dauert eine durchschnittliche, komplexe Prozedur inklusive Splitting und Testen?“
 - **Probe B (Komplexität):** „Wie groß ist die Streuung? Was ist der Zeitunterschied zwischen einem simplen \$?=-Umschreiben und Logiken mit verschachtelten IF-Bedingungen?“
 - **Probe C (Gesamtprojekt):** „Wenn ihr ein gesamtes Kundenprojekt betrachtet: Welcher prozentuale Anteil der Projektzeit entfällt rein auf diese syntaktische und logische Code-Migration?“

Phase 3: Kognitiver Aufwand & Fehlerquellen (ca. 10 Minuten)

(Regieanweisung: Argumente sammeln, warum der Mensch fehleranfällig ist und eine KI-gestützte Automatisierung (das Artefakt) einen Mehrwert bietet.)

- **Hauptfrage 4 (Offen):** „Wenn du auf die letzten Migrationen zurückblickst: Was empfindest du bei diesem manuellen Umschreiben als die größte kognitive Herausforderung? Wo raucht der Kopf am meisten?“
 - **Probe A (Fehlerarten):** „Welche Fehler passieren den Kollegen oder dir am häufigsten, die dann später im Test auffallen (z. B. vergessene Punkte, falsch aufgelöste IF-Bedingungen)?“
 - **Probe B (Qualitätssicherung):** „Wie stellt ihr aktuell sicher, dass der migrierte AVC-Code logisch exakt dasselbe tut wie der alte LO-VC-Code? Gibt es da automatisierte Verfahren oder ist das rein manuelles Testen?“

Phase 4: Abschluss & Blick auf die Lösung (ca. 5 Minuten)

(Regieanweisung: Überleitung von der Problemstellung (Baseline) zur Lösungsbewertung (KI-System).)

- **Hauptfrage 5:** „Angenommen, wir hätten ein System, das dir dieses Splitting und die Syntax-Übersetzung zu 90 % fehlerfrei automatisiert abnimmt: Welchen Einfluss hätte das auf deinen Arbeitsalltag und die Projektlaufzeiten?“
- **Abschluss:** „Gibt es aus deiner Sicht noch einen wichtigen Aspekt beim manuellen Umschreiben von LO-VC, über den wir heute noch gar nicht gesprochen haben?“

A.2 Leitfaden Evaluation

Phase 1: Das Briefing (Die ersten 5 Minuten)

Hier werden die methodischen Rahmenbedingungen gesetzt. Die wichtigsten Fragen hierfür:

- „Darf ich diesen Call für meine Masterarbeit aufzeichnen, damit ich später deine Zitate auswerten kann?“
- „Wir machen heute ein Think-Aloud-Verfahren. Ich zeige dir gleich Excel-Dashboards, die mein Agent generiert hat.“
- „Deine Aufgabe: Bitte sprich alles laut aus, was dir durch den Kopf geht, während du die Excel liest. Auch wenn du stockst, dich wunderst oder fluchst – sag es laut.“

- „Wichtig: Ich bin heute nur neutraler Beobachter. Ich werde keine Architektur-Entscheidungen der KI verteidigen oder dir erklären, warum sie etwas falsch gemacht hat. Ich höre nur zu.“

Phase 2: Die Think-Aloud Durchführung (ca. 30–40 Minuten)

Die Excel Dateien zusammen mit den JSON Files wurden im Vorhinein geschickt. Der Experte liest sich durch die Dateien. Wenn der Experte verstummt, können folgende Ermutigungen helfen, seine Gedanken zu teilen:

- **Wenn er länger als 15 Sekunden schweigt:** „Was schaust du dir gerade an?“ oder „Was denkst du gerade?“
- **Wenn er stutzt oder seufzt:** „Warum hast du gerade gezögert?“
- **Wenn er sagt: „Das ist ja komisch“:** „Was genau ist daran komisch?“
- **Wenn er sagt: „Das hätte ich anders gemacht“:** „Wie hättest du es manuell gelöst?“

Phase 3: Das Systematische Debriefing (ca. 10 Minuten)

Wenn alle 5 Dashboards durchgesehen sind, wird der Think-Aloud-Modus für gezielte Rückfragen beendet. Diese Fragen decken die in Kapitel 7 definierten Evaluierungskriterien ab:

Kriterium 1: Nachvollziehbarkeit (Traceability)

„Wie gut konntest du anhand der Notizen des ‘Architekten’ nachvollziehen, warum aus dem alten Code eine Tabelle oder ein IF-Statement wurde?“

Kriterium 2: Semantische Äquivalenz (Logik)

„Hat das Multi-Agenten-System die eigentliche Geschäftslogik der Vorbedingungen verfälscht, oder blieb die fachliche Bedeutung bei der Migration erhalten?“

Kriterium 3: Syntaktische Korrektheit (AVC-Standard)

„Wie bewertest du die rohe Syntax, die der Synthesizer am Ende ausgespuckt hat? Was waren die größten Schnitzer (z. B. fehlendes Präfix) und wie schlimm sind diese für die Praxis?“

Abschlussfrage (Business Value):

„Glaubst du, dass ein Entwickler mit diesem Dashboard als Vorlage schneller wäre als bei einer komplett händischen Migration, selbst wenn er syntaktische Fehler noch manuell korrigieren muss?“

Anhang B

Codebücher der qualitativen Analyse

B.1 Codebuch Experteninterview

Code / Kategorie	Definition	Ankerbeispiel (aus dem Interview)
RQ2-1: 01_Manuel- ler_Prozess	Erfassung aller händischen Schritte, Systemwechsel und Prozessabhängigkei- ten.	
RQ2-5: 01_01_Prozess_ Schritt_Manuell	Jede explizite Nennung einer händischen Tätigkeit, die ein Entwickler bei der Übertragung von LO-VC zu AVC physisch ausführen muss.	„Wir legen die Struktur für die Tabelle dann im AVC komplett händisch neu an.“

Fortsetzung auf der nächsten Seite...

Tabelle B.1 (Fortsetzung)

Code / Kategorie	Definition	Ankerbeispiel (aus dem Interview)
RQ2-6: 01_02_Prozess_ Toolbruch	Medienbrüche, bei denen das SAP-System verlassen und Drittsoftware (insbesondere Excel) für Workarounds genutzt werden muss.	„Da müssen wir uns die Daten erst in ein Excel ziehen und dort bereinigen, bevor wir sie wieder hochladen.“
RQ2-2: 02_Zeitlicher_Aufwand	Identifikation von Flaschenhälsen und Extraktion harter Metriken für die Baseline.	
RQ2-8: 02_01_Zeit_ Dauer_Konkret	Nennung harter numerischer Werte und Zeiträume (Stunden, Tage, Monate, etc.) für spezifische Migrationsschritte oder Gesamtprojekte.	„Für ein durchschnittliches Modell brauchen wir aktuell etwa drei Personentage.“
RQ2-9: 02_02_Zeit_ Flaschenhals	Subjektive Einschätzung des Experten, welche Aufgabe den unverhältnismäßig größten Zeitanteil frisst oder den Prozess am stärksten bremst.	„Die meiste Zeit verlieren wir eigentlich bei der Bereinigung der alten Tabellen.“

Fortsetzung auf der nächsten Seite...

Tabelle B.1 (Fortsetzung)

Code / Kategorie	Definition	Ankerbeispiel (aus dem Interview)
RQ2-3: 03_Kognitiver_Aufwand	Erfassung dessen, was das menschliche Gehirn überlastet und wo typische Fehler passieren.	
RQ2-11: 03_01_Kognitiv_ Altlasten	Schwierigkeiten bei der Migration, die nicht durch das neue System (AVC) entstehen, sondern durch das Verstehen von historisch gewachsenem, undokumentiertem oder totem Code im Altsystem (LO-VC).	„Das Problem ist Code, der 15 Jahre alt ist und bei dem niemand mehr weiß, ob der überhaupt noch genutzt wird.“
RQ2-12: 03_02_Kognitiv_ Syntax_Umdenken	Kognitive Schwierigkeiten, die durch die neuen technischen Paradigmen, die strengere Syntax oder die HANA-Logik von AVC entstehen.	„In AVC musst du die Logik teilweise völlig anders aufbauen, das alte LO-VC Denken funktioniert da nicht mehr.“
RQ2-13: 03_03_Kognitiv_ Fehler_Menschlich	Typische Flüchtigkeits- oder Konzentrationsfehler, die Entwicklern aufgrund der monotonen, manuellen Arbeit unterlaufen.	„Wenn du das stundenlang abtippst, vergisst du halt auch einfach mal eine Klammer oder ein Komma.“

Fortsetzung auf der nächsten Seite...

Tabelle B.1 (Fortsetzung)

Code / Kategorie	Definition	Ankerbeispiel (aus dem Interview)
RQ2-4: 04_Tool_Anforderungen	Direkte Lösungsansätze und Wünsche des Experten für die KI-Agenten.	
RQ2-14: 04_01_Tool_Konkrete_Anforderung	Ein direkter, konkreter Funktionswunsch an eine unterstützende Software oder KI zur Bewältigung der in Kategorie 1-3 genannten Probleme.	„Das Tool müsste mir eigentlich direkt die Zeile markieren können, in der der Syntax-Fehler liegt.“
RQ2-15: 04_02_Tool_Usability	Anforderungen an die Transparenz, Bedienbarkeit und das Vertrauen in ein automatisiertes System (Explainability).	„Die KI darf keine Blackbox sein, ich muss als Entwickler nachvollziehen können, warum sie das Constraint so umgeschrieben hat.“
RQ2-16 (induktiv): 05_01_Explizite_Tool_Nennung	Explizite Nennung von spezifischen Software-Tools, die im Prozess genutzt werden oder fehlen.	– (<i>Induktiv erhoben</i>)
RQ2-17 (induktiv): 05_02_Testen	Thematisierung des Aufwands und der Vorgehensweise bei der Validierung und beim Testen der Modelle.	– (<i>Induktiv erhoben</i>)

Fortsetzung auf der nächsten Seite...

Tabelle B.1 (Fortsetzung)

Code / Kategorie	Definition	Ankerbeispiel (aus dem Interview)
RQ2-18 (induktiv): 05_03_Komplexität_Modellgröße	Aussagen zur Skalierbarkeit, Größe und strukturellen Komplexität der zu migrierenden Produktmodelle.	– (<i>Induktiv erhoben</i>)
RQ2-19 (induktiv): 05_04_Limitierung_KI_Tool	Erwähnung von Grenzen, Schwächen oder Herausforderungen beim potenziellen Einsatz von KI-Tools.	– (<i>Induktiv erhoben</i>)

B.2 Codebuch Evaluation

Code / Kategorie	Definition	Ankerbeispiel
RQ4-1: Nachvollziehbarkeit	Alles, was sich darauf bezieht, wie gut der Gedankengang des Architekten lesbar, granular und logisch dokumentiert ist.	„Der Experte kritisierte, dass die architektonische Herleitung des Agenten nicht immer schlüssig war. Zwar deckte sich die Entscheidung <code>NO_TABLE</code> mit dem Fehlen der Tabelle im Code, das Warum fehlte jedoch. Bei komplexen Bedingungen konnte der Experte nicht nachvollziehen, warum das System sich gegen eine Tabelle entschieden hat.“
RQ4-2: Semantische Äquivalenz & Logik (Struktur & Architektur)	Alles, was sich auf den inhaltlichen und logischen Aufbau der Tabellen bezieht (z. B. der richtige oder falsche architektonische Umgang mit UNCLASSIFIED-Merkmalen und das Fehlen des Wildcard-Konzepts).	„Das System hat Werte ignoriert, die keine Vorbedingung hatten, anstatt zu erkennen, dass diese Werte immer gültig sind. Zudem hat es bei redundanten Bedingungen zwei separate Tabellen erstellt, anstatt eine einzige Tabelle mit der Schnittmenge und Wildcards (Platzhaltern) zu bauen.“

Fortsetzung auf der nächsten Seite...

Tabelle B.2 (Fortsetzung)

Code / Kategorie	Definition	Ankerbeispiel
RQ4-3: Syntaktische Korrektheit (AVC-Compliance)	Alles, was harte SAP-Regeln, Syntaxfehler, Präfixe oder fehlende Zeichen im generierten Code betrifft.	„Das System hat den Code NOT SPECIFIED(UNCLASSIFIED_umgebung) generiert. Der Experte merkte an, dass dies ein Hard-Fail ist, da die neue AVC-Engine NOT SPECIFIED in Constraints syntaktisch nicht mehr zulässt. Auch der Syntax-Aufruf unterhalb der korrekten Tabellenmatrizen war fehlerhaft formatiert.“
RQ4-4: Praxistauglichkeit & Business Value	Alles, was die finale Frage beantwortet, ob das Tool dem Entwickler Arbeit abnimmt, Zeit spart oder in einen produktiven Workflow integriert werden kann.	„In der exakten jetzigen Form spart das Dashboard noch keine Zeit, da die manuelle Nacharbeit bei Wildcards zu hoch ist. Allein das automatische Erkennen der nicht klassifizierten Merkmale liefert aber bereits einen enormen inhaltlichen Mehrwert für den menschlichen Bearbeiter.“

Anhang C

Dokumentation Selbstversuch

Artefakt 1: K40__BRM

Ausgangslage (LO-VC Input / Rohdaten):

- **Merkmalsname:** K40__BRM (Charakter-Merkmal).
- **Zugeordnete Werte:** 0, 1 und 2.
- Wert 0 hat keine Vorbedingungen.
- Wert 1 hat die Vorbedingungen: V_TW0003 (K40_fg_af = '3') und V_001 (specified dummy).
- Wert 2 hat die Vorbedingung: V_TW0004 (K40_fg_af = '1').

Meine Analyse & Gedankengänge bei der Migration:

- **Logik-Verknüpfung:** Wenn ein Wert mehrere Preconditions hat, müssen alle erfüllt sein (logisches AND).
- **Umgang mit „specified dummy“:** Ich weiß aus Erfahrung mit den Kundendaten, dass der Ausdruck `specified dummy` immer dann verwendet wird, wenn ein Wert nie aufgerufen werden soll (da der Dummy nie klassifiziert wird). Das ist quasi historisch gewachsener „toter Code“. **Meine Entscheidung:** Ich lasse Wert 1 komplett bei der Migration weg.
- **Struktur-Entscheidung:** Da Wert 0 keine Bedingung hat und Wert 1 gelöscht wird, verbleibt nur noch Wert 2 mit einer Bedingung. Da es nur ein einziger Wert ist, schreibe ich das als direkte Code-Zeile und schränke es nicht über eine extra Variantentabelle ein.

- **AVC-Syntax-Anpassung:** Ich war mir anfangs unsicher, wie der genaue AVC-Code aussehen soll. Nach Blick in die Dokumentation lasse ich den alten INFERENCES-Block komplett weg, da AVC ohnehin alles herleitet. Auch den umständlichen OBJECTS-Block spare ich mir und referenziere die Merkmale direkt über das Präfix `$self..`

Mein erstellter AVC Ziel-Code:

RESTRICTIONS:

```
$self.K40_BRM = '2' if $self.K40_fg_af = '1'.
```

Artefakt 2: K40__KO

Ausgangslage (LO-VC Input / Rohdaten):

- **Merkmalsname:** K40__KO mit den Werten 01 bis 10 sowie 99.
- Viele Werte haben komplexe, teils redundante Vorbedingungen im JSON-Format hinterlegt.

Meine Analyse & Gedankengänge bei der Migration:

- **Bereinigung toter Code:** Die Werte 04, 06, 09 und 10 haben wieder die Vorbedingung `specified dummy`. Diese ignoriere ich bei der Ziel-Generierung komplett.
- **Bereinigung Kommentare:** Bei Wert 99 sind drei der vier Zeilen der Vorbedingung mit einem Stern (*) auskommentiert. Es verbleibt für diesen Wert also eigentlich nur eine echte, aktive Vorbedingung.
- **Mustererkennung & Tabellenbildung:** Mir fällt auf, dass sich die Werte 01, 02 und 03 exakt dieselbe Tabellen-Vorbedingung teilen. Das Gleiche gilt für die Werte 05 und 07. Wenn ich das alles als einzelne IF-Zuweisungen ausschreibe, wird der Code endlos lang. Ich entscheide mich, diese Cluster in zwei Variantentabellen zusammenzufassen.
- **Syntax-Update:** Wert 08 hat eine sehr eigenständige Vorbedingung, in der `ne` (not equal) vorkommt. Die korrekte Syntax hierfür in der modernen AVC-Engine ist `<>`. Das muss ich beim Schreiben ersetzen.

Mein erstellter AVC Ziel-Code:

RESTRICTIONS:

```
table K40_KO_01 (
    K40_fg_af = $self.K40_fg_af,
```



```

    K40_abl = $self.K40_abl,
    K40_K0 = $self.K40_K0
),
table K40_K0_02 (
    K40_fg_af = $self.K40_fg_af,
    K40_ts = $self.K40_ts,
    K40_K0 = $self.K40_K0
),
$self.K40_K0 = '08' if ($self.K40_fg_af = '3' or $self.K40_fg_af = '4')
    and $self.K40_brm = '0' and $self.K40_ts <> '1',
$self.K40_K0 = '99' if $self.K40_pl in ('d1','h1','h2').

```

Meine Tabellen hierzu:

- Für die erste Variantentabelle (K40_KO_01) werden die Werte 01, 02 und 03 zusammengefasst. Aus dem JSON geht hervor, dass diese Werte exakt dieselbe Vorbedingung teilen (K40_fg_af in ('1','2') und K40_abl = '0'). Aufgelöst in eine Tabellenmatrix ergeben sich daraus insgesamt sechs gültige Kombinationen (Zeilen): Die Zielwerte 01, 02 und 03 werden jeweils einmal mit der Merkmalskombination K40_fg_af = 1 & K40_abl = 0 sowie ein weiteres Mal mit K40_fg_af = 2 & K40_abl = 0 verknüpft.
- Für die zweite Variantentabelle (K40_KO_02) werden die Zielwerte 05 und 07 analog behandelt. Da in deren JSON-Vorbedingung zwei IN-Operatoren aufeinandertreffen (K40_fg_af in ('3','4') und K40_ts in ('0','1')), entstehen pro Zielwert vier Kombinationsmöglichkeiten. Daraus ergeben sich insgesamt acht Tabellenzeilen: Die Zielwerte 05 und 07 werden jeweils mit allen vier möglichen Ausprägungen der Vorbedingungen verknüpft (K40_fg_af = 3 oder 4, kombiniert mit K40_ts = 0 oder 1).

Artefakt 3: K40_PL

Ausgangslage (LO-VC Input / Rohdaten):

- **Merkmalsname:** K40_PL (Char).
- **Zugeordnete Werte:** H2, H3, H5, D1, H1, H4, D2.
- Werte H2, H3 und H5 enthalten alle die Bedingung **specified dummy**. Bei H2 steht diese in Kombination mit einer weiteren Bedingung (K40_fst = '0').
- Werte D1 und H1 haben beide exakt dieselbe Bedingung: K40_fst = '0'.

- Wert H4 hat die Bedingung: `Specified K40_FG_AF and K40_FG_AF = '4'`.
- Wert D2 hat keine Vorbedingungen.

Meine Analyse & Gedankengänge bei der Migration:

- **Umgang mit Dummy-Werten:** Auch hier taucht das Anti-Pattern `specified dummy` wieder auf. Die Werte H3 und H5 fliegen direkt raus. Interessant ist H2: Hier gibt es noch eine zweite Vorbedingung. Da Vorbedingungen bei Werten aber logisch mit AND verknüpft werden, zieht der Dummy den kompletten Wert mit runter. Ich lösche H2 also ebenfalls komplett aus der Ziel-Architektur.
- **Redundanz-Bereinigung:** Bei Wert H4 steht `Specified K40_FG_AF and K40_FG_AF = '4'`. Das ist logisch betrachtet „doppelt gemoppelt“. Wenn ein Merkmal den Wert '4' hat, ist es automatisch auch spezifiziert. Ich entscheide mich, diesen alten Ballast abzuwerfen und den AVC-Code schlanker zu machen, indem ich das `Specified` einfach weglasse.
- **Mustererkennung & Tabellenbildung:** Die Werte D1 und H1 teilen sich wieder exakt dieselbe Bedingung (`K40_fst = '0'`). Um den Code elegant zu halten und die Engine nicht mit IF-Statements zu überladen, packe ich diese beiden Werte in eine kleine Variantentabelle.
- **Wert ohne Bedingung:** Da D2 keine Bedingung hat, muss ich dafür keinen Code generieren. Er ist einfach immer erlaubt.

Mein erstellter AVC Ziel-Code:

RESTRICTIONS:

```
table K40_PL_01 (
    K40_fst = $self.K40_fst,
    K40_PL = $self.K40_PL
),
$self.K40_PL = 'H4' if $self.K40_FG_AF = '4'.
```

Meine Tabellen hierzu:

- **Tabelle 1 (K40_PL_01):**
 - Spalten: `K40_fst`, `K40_PL`
 - Zeilen: 0, D1; 0, H1

Artefakt 4: K40_SCHP

Ausgangslage (LO-VC Input / Rohdaten):

- **Merkmalsname:** K40_SCHP (Char) mit den Werten 01 bis 10.
- **Besonderheiten bei den Vorbedingungen:**
 - Wert 02: Enthält einen großen Block an auskommentiertem Freitext (z. B. *Merkmalswert ausgeblendet) und nur eine aktive Zeile (not specified K40_FG_AF).
 - Werte 01 und 10: Haben exakt dieselbe Bedingung (K40_ko_af = '1' and K40_ps ne 'pw04' and K40_ps ne 'ps03').
 - Werte 07 und 08: Haben exakt dieselbe Bedingung (K40_ko_af = '2' and K40_fg_af in ('3','4') and K40_ko ne '07').
 - Werte 03 und 09: Haben exakt dieselbe Bedingung (K40_ko = '08' and K40_abl = '0').
 - Wert 05: Ähneln 03/09, prüft aber auf K40_ko = '09'.

Meine Analyse & Gedankengänge bei der Migration:

- **Code-Bereinigung:** Bei Wert 02 hat früher offensichtlich jemand die Dokumentation direkt in den Code geschrieben und alte Logik auskommentiert (*). Das muss rigoros gelöscht werden, es zählt nur die aktive Zeile.
- **Syntax-Update:** Der veraltete Operator **ne** taucht in mehreren Bedingungen auf. Dieser muss zwingend in **<>** übersetzt werden.
- **Struktur-Entscheidung (Redundanzen bündeln):** Es gibt hier massive Doppelungen. Da einige Bedingungen Ungleichungen (**ne** bzw. **<>**) enthalten, eignen sich diese nicht optimal für klassische Variantentabellen (da Tabellen meist auf reine Gleichheit prüfen). Daher entscheide ich mich hier für einen anderen Weg der Bündelung: Ich schreibe die Restriktionen nicht einzeln pro Wert, sondern fasse die Werte im IF-Statement über Listen-Abfragen (**in**) zusammen. Das hält den Code ebenfalls sehr kompakt und ist für die AVC-Engine schnell auswertbar.

Mein erstellter AVC Ziel-Code:

RESTRICTIONS:

```
$self.K40_SCHP = '02' if not specified $self.K40_FG_AF,  
$self.K40_SCHP = '06' if $self.K40_brm = '0' and $self.K40_abl = '0',  
$self.K40_SCHP = '04' if $self.K40_ko = '09' and not specified $self.K40_ps,  
$self.K40_SCHP in ('03', '09') if $self.K40_ko = '08' and $self.K40_abl = '0',
```

```

$self.K40_SCHP = '05' if $self.K40_ko = '09' and $self.K40_abl = '0',
$self.K40_SCHP in ('01', '10') if $self.K40_ko_af = '1'
    and $self.K40_ps <> 'pw04' and $self.K40_ps <> 'ps03',
$self.K40_SCHP in ('07', '08') if $self.K40_ko_af = '2'
    and $self.K40_fg_af in ('3','4') and $self.K40_ko <> '07'.

```

Artefakt 5: K40_GK_FB

Ausgangslage (LO-VC Input / Rohdaten):

- **Merkmalsname:** K40_GK_FB (Char) mit insgesamt 10 Werten (Farb- bzw. Strukturcodes wie 1018, 3000, 2004 etc.).
- **Besonderheit:** Die ersten 7 Werte (1018, 3000, 5002, 5003, 6024, 2002, 7016) haben alle exakt dieselbe Vorbedingung hinterlegt: V_TW0228 mit der Syntax K40_grh_af ne '3'.
- Die verbleibenden 3 Werte (2004, 7032, 7035) besitzen überhaupt keine Vorbedingungen.

Meine Analyse & Gedankengänge bei der Migration:

- **Mustererkennung & Redundanz:** Das ist ein klassisches Beispiel für ineffiziente Datenpflege im Altsystem. Anstatt die Bedingung einmal zentral abzuprüfen, wurde sie an 7 verschiedene Einzelwerte gehängt.
- **Struktur-Entscheidung (Architektur-Wahl):** Wenn ich das 1:1 in AVC übersetze, müsste ich ein IF-Statement mit einer langen Liste (IN ('1018', '3000', ...)) schreiben. Da es hier aber ausschließlich um eine einfache Bedingung mit einer Ungleichung (ne) geht und die anderen drei Werte ohnehin immer gelten, entscheide ich mich für eine elegantere Logik-Umkehr. Anstatt positiv aufzulisten, welche 7 Werte bei <> '3' erlaubt sind, schreibe ich ein Constraint, das definiert, was passiert, wenn der Wert doch '3' ist (dann sind eben nur noch die drei freien Werte erlaubt).
- **Syntax-Update:** Der Operator ne wird in AVC durch <> ersetzt.

Mein erstellter AVC Ziel-Code:

RESTRICTIONS:

```

$self.K40_GK_FB in ('2004', '7032', '7035') if $self.K40_grh_af = '3'.

```

Anhang D

Beispiel

D.1 Beispielhafter JSON Input

Merkmal: K40_BRM

Beschreibung: K40_BRM

Typ: char

- **Wert 1** (Beschreibung: 1)
 - *Bedingung (V_TW0003):* K40_fg_af = '3'
 - *Bedingung (V_001):* specified dummy
- **Wert 2** (Beschreibung: 2)
 - *Bedingung (V_TW0004):* K40_fg_af = '1'
- **Wert 0** (Beschreibung: 0)

Merkmal: K40_KO

Beschreibung: K40_KO

Typ: char

- **Wert 04** (Beschreibung: 04)
 - *Bedingung (V_001):* specified dummy
 - *Bedingung (V_TW0008):* K40_fg_af in ('1','2')
 - *Bedingung (V_TW0569):* K40_ab1 = '0'
- **Wert 06** (Beschreibung: 06)
 - *Bedingung (V_001):* specified dummy

- *Bedingung* (*V_TW0009*): K40_fg_af in ('3','4') and K40_ts in ('0','1')
- **Wert 09** (Beschreibung: 09)
 - *Bedingung* (*V_001*): specified dummy
 - *Bedingung* (*V_TW0011*): K40_fg_af = '1' and K40_brm = '0' and K40_ts ne '1' and K40_pl_af = 'd' and K40_pl in ('d1','d2') (K40_pl in('d1','d2') or K40_pl in ('H1','H3'))
 - *Bedingung* (*V_TW0569*): K40_abl = '0'
- **Wert 10** (Beschreibung: 10)
 - *Bedingung* (*V_001*): specified dummy
- **Wert 01** (Beschreibung: 01)
 - *Bedingung* (*V_TW0008*): K40_fg_af in ('1','2')
 - *Bedingung* (*V_TW0569*): K40_abl = '0'
- **Wert 02** (Beschreibung: 02)
 - *Bedingung* (*V_TW0008*): K40_fg_af in ('1','2')
 - *Bedingung* (*V_TW0569*): K40_abl = '0'
- **Wert 03** (Beschreibung: 03)
 - *Bedingung* (*V_TW0008*): K40_fg_af in ('1','2')
 - *Bedingung* (*V_TW0569*): K40_abl = '0'
- **Wert 05** (Beschreibung: 05)
 - *Bedingung* (*V_TW0009*): K40_fg_af in ('3','4') and K40_ts in ('0','1')
- **Wert 07** (Beschreibung: 07)
 - *Bedingung* (*V_TW0009*): K40_fg_af in ('3','4') and K40_ts in ('0','1')
- **Wert 08** (Beschreibung: 08)
 - *Bedingung* (*V_TW0010*): (K40_fg_af = '3') or K40_fg_af = '4' and K40_brm = '0' and K40_ts ne '1'
- **Wert 99** (Beschreibung: 99)

– *Bedingung (0000003046):* *K40_fg_af in ('1','2','3','4') and
 K40_pl in ('d1','h1','h2')
 K40_fg_af in ('1','2','3','4') and
 K40_pl in ('d2','h3') and K40_fst = '0'

D.2 Erstes Beispiel eines Ergebnisses der V1-Architektur

```
/* =====
AVC CONSTRAINT FUER MERKMAL: K40_BRM
===== */
TABLE K40_BRM
DESCRIPTION 'K40_BRM Characteristics'
CODE 'K40_BRM'
{
  '1': '1',
  RESTRICT ($self.K40_fg_af='3' AND $self.V_TW0003);

  '2': '2',
  RESTRICT ($self.K40_fg_af='1' AND $self.V_TW0004);

  '0': '0'
}

/* =====
AVC CONSTRAINT FUER MERKMAL: K40_KO
===== */
TABLE K40_KO
CONSTRAINT RESTRICT (
  WHEN $self.code IN ('01', '02', '03', '04') THEN
    $self.dependency_precondition IN ('V_TW0008', 'V_TW0569')
    AND
    $self.dependency_syntax IN ($self.K40_fg_af IN ('1','2'),
    $self.K40_abl = '0')
  WHEN $self.code IN ('05', '07') THEN
    $self.dependency_precondition = 'V_TW0009' AND
    $self.dependency_syntax IN ($self.K40_fg_af IN ('3','4')
    AND $self.K40_ts IN ('0','1'))
  WHEN $self.code = '08' THEN
    $self.dependency_precondition = 'V_TW0010' AND
```

```

$self.dependency_syntax IN (($self.K40_fg_af = '3') OR
    $self.K40_fg_af = '4' AND $self.K40_brm = '0' AND NOT
    $self.K40_ts = '1')
WHEN $self.code = '09' THEN
    $self.dependency_precondition IN ('V_TW0011', 'V_TW0569')
    AND
    $self.dependency_syntax IN ($self.K40_fg_af = '1' AND $self
        .K40_brm = '0' AND NOT $self.K40_ts = '1' AND $self.
        K40_pl_af = 'd' AND ($self.K40_pl IN ('d1','d2') OR $self
        .K40_pl IN ('H1','H3')), $self.K40_abl = '0')
WHEN $self.code = '99' THEN
    $self.dependency_precondition = '0000003046' AND
    $self.dependency_syntax IN ($self.K40_fg_af IN
        ('1','2','3','4') AND $self.K40_pl IN ('d1','h1','h2') OR
        $self.K40_fg_af IN ('1','2','3','4') AND $self.K40_pl IN
        ('d2','h3') AND $self.K40_fst = '0')
);

```

D.3 Erstes Beispiel des Ergebnisses der finalen Architektur

Das Finale Ergebnis der V4-Architektur ist ein Excel-Dashboard, das die migrierten Merkmale, die Tabellenvorschläge und den zugehörigen AVC-Code in einer übersichtlichen Form darstellt. Die nachfolgende Abbildung zeigt einen Teilausschnitt des Dashboards für das erste Merkmal K40_BRM.

MAS MIGRATIONS-VORSCHLAG: K40_BRM

1. Herleitung (Gedankengang des Architekten)

Der Bauplan gibt an, dass die Entscheidung 'TABLE' ist, daher muss ein Tabellenaufwurf generiert werden. Die Bedingungsspalte ist 'K40_FG_AF', und die Zielspalte ist 'K40_BRM'. Die Zeilen-Daten geben die Zuordnung der Werte basierend auf der Bedingungsspalte an. Die 'original_code_mapping' zeigt die Logik, die für die Zuweisung verwendet wird, aber da die Entscheidung 'TABLE' ist, wird diese Logik nicht direkt in IF-Bedingungen übersetzt, sondern in einen Tabellenaufwurf integriert.

2. Visuelle Variantentabelle (Für CU60)

K40_FG_AF	K40_BRM
3	1
1	2

3. Generierter AVC Constraint Code

TABLE VC TAB_NAME (K40_FG_AF = K40_FG_AF).

Abbildung D.1: Ausschnitt des generierten Excel-Dashboards (V4-Output)